University of Ljubljana
Faculty of Computer and Information Science

Martin Špendl

Self-supervised Learning of Cox Regression for Latent Gene Set Representation

master's thesis

the 2nd cycle master's study programme
computer and information science
Track: Data Science

Supervisor: Prof. Dr. Blaž Zupan

Ljubljana, 2024

Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Martin Špendl

Samospodbujevano učenje Coxovega regresijskega modela za predstavitev genskih skupin v latentnem prostoru

magistrsko delo

magistrski študijski program druge stopnje
računalništvo in informatika
Smer: Podatkovne Vede

Mentor: prof. dr. Blaž Zupan

Ljubljana, 2024

Acknowledgments

I would like to acknowledge my mentor, Prof. Dr. Blaž Zupan, for his guidance during the research and writing of this thesis. I also thank Assoc. Prof. Dr. Tomaž Curk for all the discussions and advice during the writing of the article and the thesis. Many thanks also to all the members of the bioinformatics laboratory, who made the research process incredibly enjoyable.

I would also like to thank my family and friends for their support and encouragement. Above all, I am deeply grateful to my fiancée Tina, who made this long journey wonderful. And Po.

Martin Špendl, 2024

Contents

Abstract

Povzetek

Razširjeni povzetek **i**

I Kratek pregled sorodnih del ii

II Predlagana metoda iii

III Ovrednotenje na kliničnih podatkih iv

IV Sklep vi

1 Introduction **1**

1.1 Related work 2

1.2 Problem definition 3

1.3 Our contribution 5

2 Theoretic Background **7**

2.1 Foundations of Molecular Biology 7

2.2 Machine Learning 10

2.3 Survival Analysis 15

3 Decomposition of transcription and mRNA decay **27**

3.1 First-order dynamics of mRNA concentration 28

3.2 Modeling mRNA concentration with survival analysis 30

3.3 Time-differentiable CoxPH partial likelihood 31

3.4 Transcription-decay decomposition loss 33

3.5 Implementation details 34

4 Evaluation method **41**

4.1 Data 41

4.2 Inference of latent representations 44

4.3 Evaluation on downstream tasks 45

5 Results **49**

5.1 TDD improves cancer subtype clustering over MSE 49

5.2 TDD achieves the highest performance across drug response predictions 51

5.3 Different latent representations have no significant effect on survival modeling 52

5.4 Discussion 53

6 Conclusions **57**

Abstract

Title: Self-supervised Learning of Cox Regression for Latent Gene Set Representation

Enhanced understanding of disease mechanisms on a molecular level leads to more effective treatment decisions. The high-dimensional nature of such data requires dimensionality reduction techniques to extract important patterns. To improve the interpretation and relevance of latent representations, domain knowledge is introduced during modeling by influencing data preprocessing or model architecture, while domain-inspired loss functions are scarcely explored.

We propose a novel autoencoder loss function for modeling mRNA concentration based on the first-order differential equation of mRNA dynamics. We decompose the concentration into transcription (synthesis) and mRNA decay and reformulate the problem as survival analysis. By extending the definition of CoxPH partial likelihood, we perform gradient descent through both risk and survival time which achieves the correct interpretation of both processes. Representations of clinical samples and cell-line data show increased performance on clustering and drug response prediction tasks compared to standard variational autoencoders.

Keywords

variational autoencoders, mRNA dynamics, dimensionality reduction, survival analysis

Povzetek

Naslov: Samospodbujevano učenje Coxovega regresijskega modela za predstavitev genskih skupin v latentnem prostoru

Izboljšano razumevanje mehanizmov bolezni na molekulski ravni vodi do učinkovitejše terapije. Visokodimenzionalna narava takšnih podatkov zahteva uporabo tehnik zmanjšanja dimenzionalnosti, za pridobivanje ključnih vzorcev. Za izboljšano interpretacijo in relevantnost predstavitev v proces modeliranja lahko vključimo domensko predznanje, ali v pred obdelavi podatkov ali z izbiro arhitekture modela, medtem ko so vpliv domenskega znanja na cenilne funkcije še ni raziskan.

Predlagamo novo cenilno funkcijo samokodirnikov, namenjeno modeliranju koncentracije mRNA, ki temelji na diferencialni enačbi prvega reda dinamike mRNA. Koncentracijo razdelimo na transkripcijo (sintezo) in razgradnjo mRNA ter problem reformuliramo kot analizo preživetja. Z razširitvijo definicije verjetnostne porazdelitve CoxPH modela dosežemo izvedbo gradientnega spusta skozi tveganje in čas preživetja, kar omogoča pravilno interpretacijo obeh bioloških procesov. Predstavitve kliničnih vzorcev in celičnih linij dosegajo boljše rezultate pri nalogah gručenja in napovedi odziva na zdravila v primerjavi s standardnimi variacijskimi samokodirniki.

Ključne besede

variacijski samokodirnik, dinamika mRNA, zmanjševanje dimenzij, analiza preživetja

Razširjeni povzetek

Zdravljenje rakavih bolnikov je z uporabo naprednih tehnologij izjemno napredovalo, vendar so stopnje ponovitev bolezni in smrtnosti še vedno visoke [1]. Bolezen se razvije iz pacientovih hitro delečih celic, ki zaradi sprembembe DNA ali vplivov iz okolja opustijo svojo primarno funkcijo. Razumevanje osnovnih mehanizmov teh sprememb je ključnega pomena pri razvoju terapije. Glede na specifično podvrsto raka, pa lahko z uporabo bioloških označevalcev prilagodimo zdravljenje vsakemu posamezniku [2].

Najbolj razširjena metoda za meritev stanja v tumorskih tkivih je sekvenciranje mRNA (angl. *next-generation-sequencing*), ki omogoča sočasno merjenje koncentracij vseh mRNA molekul v celicah. Pri odstranjevanju šuma iz podatkov so ključnega pomena metode zmanjševanja dimenzij kot so metoda glavnih komponent (PCA) in samokodirniki (angl. autoencoders), ki poudarijo glavne vzorce in predstavijo paciente z manj značilkami. Vključevanje biološkega znanja v te metode omogoča biološko bolj relevantne predstavitve, kar vpliva na uspešnost analiz in predvsem interpretacijo [3]. Dosedanji pristopi z vključevanjem domenskega znanja obsegajo predvsem transformacije vhodnih značilk in oblikovanje nevronskih mrež na podlagi ontologije. Vključevanje biološkega predznanja v cenilne funkcije je v nasprotju z drugima pristopoma precej manj raziskano.

Meritve koncentracije mRNA pogosto obravnavamo kot časovno neodvisne, vendar dejanska koncentracija sledi procesoma transkripcije in razgradnje. Na vplive iz okolja se celice odzivajo s povečano transkripcijo ali pospešeno razgradnjo mRNA, pri čemer meritve zajamejo le trenutno sta-

nje mRNA molekul, kar lahko prikrije dinamiko teh procesov. V magistrski nalogi predstavljamo novo cenilno funkcijo samokodirnikov, ki temelji na diferencialni enačbi prvega reda dinamike mRNA. Naš pristop modelira transkripcijo in razgradnjo kot ločena procesa, pri čemer uporablja pristope analize preživetja za modeliranje dinamike. Na celičnih linijah in kliničnih podatkih smo pokazali prednosti cenilne funkcije in jo primerjali s sorodnimi pristopi.

Na podlagi magistrske naloge smo napisali konferenčni članek in ga predstavili na mednarodni konferenci Discovery Science 2024 v Pisi. Magistrsko delo vsebuje razširjen opis dela in rezultatov članka [4].

I Kratek pregled sorodnih del

Zmanjševanje števila značilk vzorcev je ključno za analizo genomskih podatkov, saj te vključujejo več deset tisoč meritev, medtem ko število pacientov v kliničnih študijah redko presega tisoč. Standardne metode, kot je PCA in njene izpeljanke, omogočajo učinkovito analizo izrazitih vzorcev v podatkih [5]. Za kompleksnejše analize, ki zahtevajo modeliranje nelinearnih interakcij, pa so bolj primerni globoki samokodirniki. Variacijski samokodirniki (VAE) vpeljejo koncept verjetnosti v latentno predstavitev podatkov, pri čemer mnogi , kot sta β-VAE [6] in odstranjevalci šuma iz podatkov [7], izboljšujejo natančnost pri analizi bioloških vzorcev [8, 9].

Glavna namena uporabe domenskega znanja pri modeliranja sta povečana razložljivost modela in njegova napovedna točnost. V gradnji nevronskih mrež so najbolj uveljavljene vidne nevronske mreže (angl. visible neural networks), pri katerih so neuroni v samokodirniku povezani glede na hierarhijo genov v bioloških procesih [10, 11]. Modeli, kot sta [12] in VEGA [13], integrirajo hierarhijo baze Gene Ontology [14] v VAE arhitekturo za natančnejšo reprezentacijo bioloških procesov. Podobni prostopi se uporabljajo tudi v modelih, ki poleg transkriptomskih podatkov vključujejo tudi druge omske podatke. Neposredno vključevanje bioloških omejitev v cenilno funkcijo pa

je še neraziskano področje, v katerega sega magistrska naloga.

II Predlagana metoda

V tem delu predlagamo novo cenilno funkcijo samokodirnikov zgrajeno na podlagi predznanja o dinamiki mRNA koncentracije. Rekonstrukcijo koncentracije na izhodu samokodirnikov smo preoblikovali v problem analize preživetja, pri čemer pojma transkripcije in razgradnje mRNA ohranita svoj biološki pomen.

Biološki podatki izraženosti mRNA

Meritve koncentracij mRNA molekul so tipično predstavljene kot matrike z nekaj sto vzorci in več kot dvajset ticoč stoplci, ki predstavljajo gene. Poleg koncentracije imajo vzorci navedene klinične podatke, kot so vrsta raka, tip tkiva, podatki o pacientu in njegovo stanje tekom raziskave. Pri delu s podatki o izraženosti smo upoštevali smernice standardne prakse. Vrednosti koncentracije smo normalizirali za vsakega pacienta, jim prišteli ena in jih transformirali z naravnim logaritmom, pri čemer smo se bolje približali normalni porazdelitvi vhodnih vrednosti. Nabor genov smo omejili na podskupino genov L1000, ki predstavljajo gene z najbolj variabilno izraženostjo.

Dinamika koncentracije mRNA molekul

Genetski material v celici predstavlja bazo osnovnih navodil o delovanju posameznih celic, medtem ko izražanje mRNA vodi v izvajanje ukazov. Vsota ukazov vseh mRNA vpliva na trenutno stanje in fenotip tkiva. Regulacija odziva celice na zunanje dejavnike se izvaja predvsem na ravni izraženosti mRNA in njene koncentracije. Koncentracija v celici je odvisna od razmerja med sintezo (transkripcije) in razpada mRNA, pri čemer lahko spremembo koncentracije zapišemo kot diferencialno enačbo prvega reda. Transkripcija in razpad mRNA v enačbi nastopata kot konstanti, ki ju lahko ob predpostavki ravnotežja med procesoma izrazimo iz enačbe. Koncentracijo lahko zapišemo kot enostavno razmerje med konstanto transkripcije in razpada ter

pretvorimo v cenilno funkcijo (DIV), vendar se konstanti pri modeliranju z samokodirniki ne obnašata v skladu z biološkim predznanjem. Da bi konstanti procesov sovpadali s predznanjem, smo se poslužili metod analize preživetja.

Modeliranje koncentracije z analizo preživetja

Analiza preživetja je veja statistike, ki obravnava podatke o času do dogodka, kot na primer od transkripcije do razpada mRNA. Prednost analize preživetja v primerjavi z regresijskim modeliranjem je sposobnost obravnavanja manjkajočih podatkov (krnjenih dogodkov) in izračuna verjetnosti v odvisnosti od vzorcev. Parametrični modeli časa do dogodka temeljijo na predpostavkah o vplivih na verjetnost dogodka tekom poteka bolezni, kar jim omogoča natančnejšo analizo z manjšim številom podatkov. Za obravnavo življenjskega cikla mRNA je najbolj primerna uporaba semi-parametričnega Coxovega modela, saj omili prej omenjene predpostavke.

Za modeliranje dinamike mRNA koncentracije smo enačbo preoblikovali v problem analize preživetja, pri katerem sedaj obravnavamo življenjski cikel posamezne mRNA molekule in verjetje življenjske dobe. Iz napovedanih konstant transkripcije in razpada mRNA izračunamo čas preživetja in tveganje za razpad mRNA (angl. risk of mRNA decay), ter uporabimo izpeljanko cenilne funkcije Coxovega modela za optimizacijo samokodirnikov (TDD). Z uvedbo časa preživetja kot distribucije in ne konstante smo izpeljaji cenilno funkcijo, ki omogoča izračun odvoda tudi preko časa preživetja. Nova uvedba spremeni definicijo setov tveganja in s tem poveča kompleksnost izračuna, vendar zagotovi biološko podprto interpretacijo konstant transkripcije in razpada mRNA molekul.

III Ovrednotenje na kliničnih podatkih

Protokol ovrednotenja

Doprinos uporabe naše cenilne funkcije glede na sorodne metode smo ovrednotili na šestih kliničnih zbirkah in obsežnem repozitoriju celičnih li-

nij. Uporabili smo predstavitve vzorcev v latentnem prostoru z uporabo samokodirnikov z različnimi cenilnimi funkcijmi (MSE, DIV, TDD), metode PCA in netransformiranega prostora vhodnih podatkov. Modele smo učili s 5-kratnim navzkrižnim preverjanjem in optimizacijo hiper parametrov. Za ovrednotenje gručenja smo uporabili oznake podtipov raka, medtem ko smo za ovrednotenje analize preživetja uporabili čas preživetja brez napredovanja bolezni po biopsiji. Odziv na zdravilo pri celičnih linijah je predstavljala koncentracija IC50, pri kateri je rast polovice celic zavrta.

Rezultati analize

Predstavitve vzorcev z našo metodo so na vseh podatkovnih zbirkah dosegle boljše rezultate pri gručenju podskupin raka kot predstavitve s klasičnimi samokodirniki. Predstavitve metode PCA so pri gručenju dosegale najvišje rezultate, kar kaže na učinkovitost izrabe nizko dimenzionalnega prostora z uporabo ortogonalnih baz. Pri odzivu na zdravila pa je poleg globalne optimizacije predstavitev pomembna tudi lokalna predstavitev. Na vzorcu 286 različnih zdravil in več kot 500 celičnih linij za posamezno zdravilo je metoda TDD dosegla najboljše rezultate na treh četrtinah zdravil. Klasična predstavitev s samokodirniki je bila boljša le od netransformiranega vhodnega prostora, pri katerem število značilk presega število vzorcev v vsah izmed podatkovnih setov. Pri analizi preživetja nismo opazili signifikantnih razlik med predstavitvami z metodami.

Skladnost rezultatov z biološkim predznanjem

Pri uporabi enostavne enačbe dinamike pri cenilni funkciji DIV opazimo visoko negativno korelacijo med konstantama transkripcije and razpada, kar je v neskladju z domenskim predznanjem. Pričakovali bi visoko korelacijo med konstantama in izraženost mRNA, ki je sorazmerna z razmerjem med obema konstantama. Pri uporabi cenilne funkcije TDD pa ohranimo željeno interpretacijo konstant bioloških procesov, pri čemer lahko izraženost mRNA modeliramo samo ob poznavanju obeh konstant. Uvedba predpostavk analize preživetja torej zagotovi potrebne omejitve prostora možnih rešitev, da

samokodirnik ohrani biološki pomen posamezne konstante.

IV Sklep

V magistrskem delu smo predstavili novo cenilno funkcijo, ki biološko znanje vključuje kot omejitve pri modeliranju. Uporaba znanja o dinamiki koncentracije mRNA v cenilni funkciji samokodirnikov prispeva k izboljšanju napovedne moči reprezentacij, zlasti gručenju pacientov in napovedovanju odziva na zdravila. To je prvi korak pri razvoju domensko vodenih metod na področju biologije, s čemer odpiramo vrata za nadaljnje raziskave. Pri nadaljnem delu želimo uporabo teh pristopov razširiti na kompleksnejšo dinamiko ter prilagoditi za obdelavo podatkov na nivoju posameznih celic. Nativno vključevanje krnjenih dogodkov v cenilni funkcije predstavlja velik potencial pri analizi izraženosti genov posameznih celic, kjer so šum in manjkajoče vrednosti prisotne v večjem delu podatkov.

Chapter 1

Introduction

During the past 30 years, there has been a tremendous improvement in cancer patient treatment; however, recurrence and mortality rates remain high [1]. Cancer evolves from patients' rapidly dividing cells that begin to deviate from their normal behavior. Understanding the root causes of this change advocates for better treatment, promoting a shift from a one-size-fits-all approach to personalized medicine [2]. The key to understanding the disease comes from discovering patterns in the genomic content of cells [2]. Next-generation sequencing (NGS) enables measuring the mRNA concentration of all molecules simultaneously but with low precision.

Distilling patterns from high-dimensional biological data heightened the need for computational approaches. Most importantly, information capture using dimensionality reduction crucially impacts performance on downstream tasks, such as patient sub-typing and drug response prediction, thus supporting better treatment-related decision-making. Injecting domain knowledge constraints into dimensionality reduction techniques can greatly impact interpretation and downstream performance [3]. However, apart from biologically inspired network architectures, little focus has been given to domain knowledge-inspired loss functions in dimensionality reduction tasks.

1.1 Related work

Data science approaches, such as dimensionality reduction methods, are crucial in combating the curse of dimensionality with more than twenty thousand gene measurements for a few hundred samples. Traditional approaches such as principle component analysis (PCA) are standard elements of mRNA expression analysis due to their efficiency and low computational cost [5]. Extensions of methods, such as supervised PCA [15] and sparse PCA [16], were introduced to improve performance; however, deep autoencoders are preferred for modeling non-linear relationships. Variational autoencoders (VAEs) are particularly renowned for their probabilistic approach to generating compressed yet comprehensive representations of data [17]. Advances in VAE latent modeling, such as β-VAE [6] and denoising [7], translate easily into the biological domain [18, 8, 9]. The single-cell variational inference (scVI) [8] framework leverages VAE for a scalable representation, addressing technical noise and bias in single-cell mRNA measurements. On the other hand, a novel method based on VAEs (scVAE) [9] forgoes conventional data preprocessing by directly modeling raw count data. There are many other VAE-based models for dimensionality reduction; however, most of them differ in either pre or postprocessing of the data. Although they achieve high performance, they lack biologically relevant constraints.

The injection of domain knowledge into model architecture represents a significant leap in enhancing the interpretability of models in biology. In their seminal papers [10, 11], Ideker et al. proposed "visible neural networks," where the architecture follows the Gene Ontology hierarchy [14] as sparsely connected layers. They aim to learn representations of biological processes for disease state and therapeutic outcome prediction, where the model's architecture is fixed and reflects the current domain knowledge. Approaches, such as OntoVAE [12] and VEGA [13], build on the visible neural networks idea by integrating it into the VAE setting. Beyond mRNA concentration measurements, multi-omics integration approaches, such as MOViDA [19] and Autosurv [20], utilize both VAE and Gene Ontology hierarchical network.

Unlike advances in model architecture, integrating domain knowledge directly into the loss function of machine learning models remains uncharted territory in biological data analysis or anywhere outside the physics domain [21]. The choice of mean squared error loss has no biological basis and, therefore, allows for solutions that might be hard to explain or are biologically meaningless. Our goal is to introduce domain knowledge in the autoencoder loss function such that additional constraints improve latent representations.

1.2 Problem definition

Measurements of mRNA concentration are usually modeled as constant over time. However, mRNA concentration follows two biological processes: transcription (synthesis of mRNA molecules from a DNA template) and decay. Cells respond to the environment by enhancing the transcription of specific mRNA molecules or enhancing decay to reverse the effect (see Fig. 1.1). Dif-

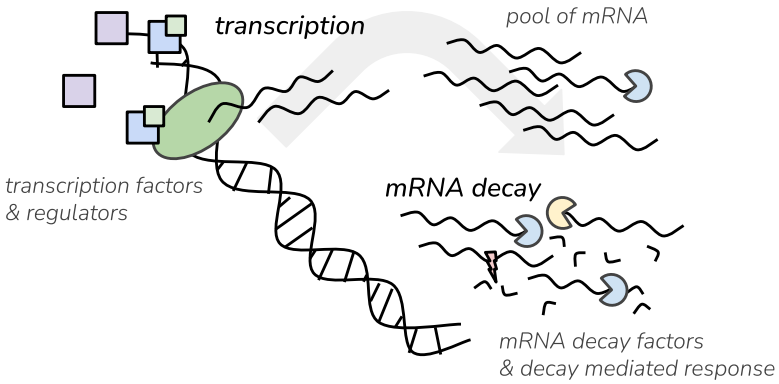

Figure 1.1: Interplay between transcription and mRNA decay. Transcription factors and other regulators regulate the synthesis of mRNA molecules in transcription. Through time, decay factors degrade mRNA molecules to regulate the homeostasis in the cell.

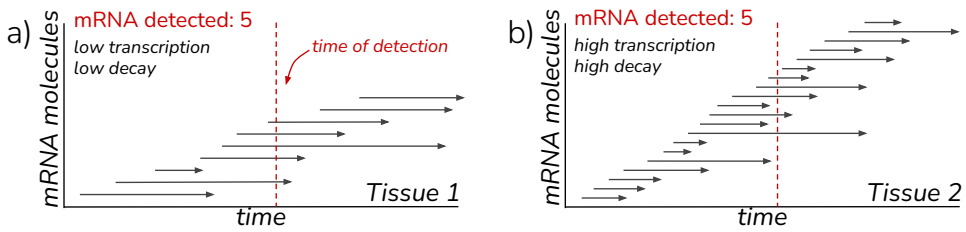

Figure 1.2: RNA-seq fails to discriminate between mRNA-dynamics scenarios. RNA-Seq experiments measure mRNA expression (the number of mRNA molecules in the pool) as a snapshot in time, thus being unable to differentiate between the **a**) low transcription-low decay and **b**) high transcription and high decay regimes. Each arrow represents the lifetime of a single mRNA molecule. The red vertical line represents the time of detection when all present mRNA molecules are counted.

ferent proteins regulate both processes with small shifts in activity leading to large changes in mRNA product.

Measurements of mRNA concentration only consider the pool of available mRNA, which can obscure the underlying dynamics. For example, a low transcription rate produces fewer mRNA molecules, but a low decay increases their lifespan (Fig. 1.2a). On the contrary, rapid transcription with a high decay rate produces the same steady-state concentration of molecules (Fig. 1.2c). In the simplest form, we can represent mRNA concentration dynamics as a first-order differential equation. However, considering the mRNA decay rate regarding molecular lifespan also allows us to use survival analysis to introduce biological constraints into dimensionality reduction approaches.

Our premise is that if we can demonstrate the benefit of the introduced loss function using plain vanilla neural networks (e.g., variational autoencoder architecture), this loss function would also be useful for other, more complex model architectures. Therefore, this thesis focuses on the introduction and exploration of the new loss function and not on the investigation of its dependency in combination with an underlying model.

1.3 Our contribution

We propose a novel transcription-decay decomposition loss function for modeling mRNA concentration based on first-order dynamics. Our approach introduces mRNA dynamics constraints directly into the loss function of the VAE as a plugin replacement for mean squared error. The loss function can be used with any autoencoder, allowing seamless integration with other domain-knowledge-guided architectures. It ensures the modeling of two vectors representing the transcription rate and the risk of mRNA decay. We construct a loss function based on the Cox proportional hazard (CoxPH) likelihood by creating a survival time proxy from those vectors. We train variational autoencoders with different losses for latent representations of patient data and evaluate the embeddings on clustering, drug-response prediction, and survival outcome.

Our paper describing our work from this thesis has recently been presented at the International Conference of Discovery Science 2024 and will be published in the conference proceedings later this year. The code to reproduce the results is available on GitHub[1].

In Chapter 2, we present the scientific background behind the core components of the thesis. Chapter 3 introduces the formulation of mRNA dynamics as a survival problem, leading to the proposal of Transcription-Decay Decomposition loss. In Chapter 4, we outline the experimental protocol, while Chapter 5 presents and discusses the results of our study. Finally, Chapter 6 provides the conclusions of the thesis.

The main contributions of the thesis are the following:

1. Theoretically formulating mRNA dynamics as a survival problem.
2. Formulating and interpreting time-distributed CoxPH likelihood.
3. Proposing and evaluating Transcription-Decay Decomposition loss for latent representation of RNA-Seq samples using autoencoders.

[1] https://github.com/MartinSpendl/DiscoveryScience24-paper

Chapter 2

Theoretic Background

Here, we present the theoretical foundation for both the biological aspects of the proposed approach and its data science fundamentals. In Section 2.1, we introduce the key biological concepts we aim to incorporate into the modeling process. In Section 2.2, we explain the theory behind variational autoencoders, the primary algorithm used in our approach. Finally, in Section 2.3, we discuss survival analysis, with a focus on the CoxPH partial likelihood and its underlying assumptions. For readers with a background in either computer science or biology who may not be familiar with some of these topics, the following sections will provide an introduction.

2.1 Foundations of Molecular Biology

Understanding cancer means understanding how tissues function in terms of specialized cell structure, function, interactions, and genetic composition. To explain high-level concepts, we must understand the hierarchy of genetic instructions in depth.

2.1.1 Cells as Building Blocks of Life

Cells are the basic units of life, the building blocks that make up every tissue and organ in an organism. At conception, cells are undifferentiated and

have no specific function. Through differentiation, they receive signals to develop into specialized cell types, such as muscle cells, neurons, or blood cells. Specialization of cells allows the formation of complex tissues with sufficient scaffolds and vascularity. A complex orchestration of tissues and connecting tissues makes up our body [22].

When cells lose control of their growth and repair mechanisms, as in cancer, this balance is disrupted, leading to disease. Understanding cells and their interactions is critical because it reveals both adaptability and vulnerability. Cell response to environmental stimuli can stretch their adaptiveness to the limits, crossing the line into pathogenic states [22].

2.1.2 The Central Dogma of Biology

In molecular biology, DNA, RNA, and proteins form the core machinery at the cellular level 2.1. DNA holds the genetic information, which cells read and translate into functional molecules through two key processes: transcription and translation. During transcription, parts of DNA are copied into RNA, which then serves as a messenger carrying instructions to the cellular machinery. In translation, RNA molecules guide the assembly of long chains of amino acids called proteins. These macromolecules perform nearly every function in a cell, from catalyzing reactions and signal processing to shaping the cell's structure. Proteins play a central role in determining how a cell behaves and interacts with its environment. Therefore, the concentration of specific proteins in the cell directly relates to cell state [22].

Proteins involved in crucial processes, such as energy production, replication, and translation, are continuously synthesized and present in steady concentrations. On the other hand, proteins required in specific response scenarios, such as viral infection, DNA damage, and extrinsic cell death, have to be produced exclusively when the signal is present. Most genes fall somewhere in the middle, where their required concentration fluctuates depending on cell and environmental signals [22].

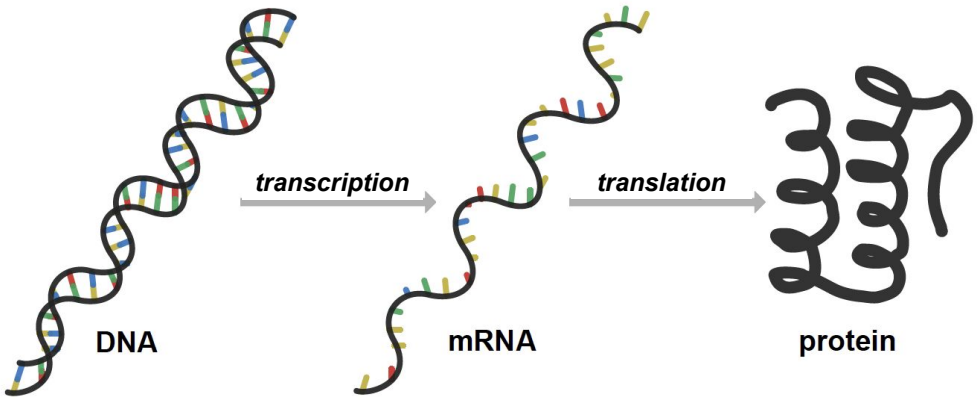

Figure 2.1: The central dogma of molecular biology. Genetic information encoded in the DNA is transcribed into RNA and then translated into amino acid sequences, forming proteins. Adjusted from [23].

2.1.3 mRNA Dynamics and Homeostasis

Transcription and translation transform genetic information into actual effector molecules and amplify the signals that invoke the response. There are usually only two copies of each gene in human cells, but thousands of proteins can be generated due to these processes. Due to their overarching influence on the cell, nature developed multiple ways of regulating these processes in the form of regulator proteins. Many proteins act as transcription regulators that activate, enhance, or inhibit transcription and translation independently. Furthermore, when mRNA molecules are synthesized, decay-related proteins start reducing their concentration, so the pool of mRNA molecules must be constantly replenished. The final layer of regulation acts on proteins and their function directly. Different forms of inhibition and activation can regulate protein activity where most behave in a concentration-dependent manner [22].

The ability to closely regulate the cell's state helps it keep the state in equilibrium, or in biological terms, in homeostasis. Homeostasis in cells refers to the ability to maintain a stable internal environment despite changing external conditions. Cells control mRNA production, degradation, and

stability to ensure that protein levels are adjusted. More extensive environmental changes, such as mutations or the presence of toxins, can push cells beyond their ability to recover homeostasis. In such scenarios, most cells undergo cell death, but some avoid it and find new homeostasis, such as cancer cells. Treating those cells with therapy means restoring healthy homeostasis or ensuring their apoptosis. Therefore, knowing the inner state of cells and tissues is crucial for effective cancer therapy [22].

2.1.4 Measuring mRNA Concentration in Tissue

One of the most used, standardized, and cost-effective methods of measuring cell states on a molecular level is RNA sequencing (RNA-Seq). The method gathers all mRNA molecules in the biological sample and reads the nucleotide sequence for molecules. By using a reference genome, sequences are mapped to their corresponding genes. The number of measured mRNA molecules for a specific gene correlates with the mRNA concentration. However, measurements represent only a snapshot in time and do not describe the dynamics of the system [24].

Novel methods of RNA-Seq are improving both the accuracy and resolution of sequencing. While the method was first introduced for tissue samples, single-cell sequencing has become widely adopted in research. In recent years, spatial transcriptomics also introduced measuring the location of the sequenced cells in addition to mRNA concentration, allowing studying of cell-cell interactions on a macroscopic scale. Furthermore, the simultaneous measuring of multiple omics is paving the way for even more complete descriptions of the biology of human cells [25].

2.2 Machine Learning

The social theory of learning states that learning is a consequence of observing the world **??**. When children mix blue and yellow paint together and get a green color, they create a mental model of how adding these colors

influences the outcome. In the same way, gathering data about our domain of interest helps us build a mathematical model and train the parameters to best represent the real world. For example, if we wanted to estimate what grade a student achieves on the exam based on the number of hours studied and the hours sleep before the exam, we could use a linear function as a model, defined as

$$grade = \beta_1 * hours_{studied} + \beta_2 * hours_{slept} + n, \tag{2.1}$$

where β_1, β_2 and n are parameters of the model. From the gathered student data on the previous exam, we would estimate the parameters of the function by maximizing the likelihood of our model. Because we know what the correct outcome of the model predictions should be (i.e. target variable), we can train the model using *supervised* learning. In reality, assigning labels (e.g., grades) to such data is costly and time-consuming. In our case, if the student grades from the previous exam were not gathered, we would have to train some other model using *unsupervised* learning. When vast amounts of unlabeled data about the domain are available, we can still employ *supervised* models. We can take some of the input features, mask them, and train the model to predict the masked features. Such training is called *self-supervised* learning. Successful examples of self-supervised models include Siamese networks in computer vision [26] and pre-trained large language models.

With ever-growing repositories of high-dimensional unlabeled data, self-supervised pre-trained models became the forefront of machine learning. Hourglass shaped neural networks called autoencoders were extensively used as a dimensionality reduction technique, with transformers taking the lead in recent years. Transformers require a substantial amount of training samples, which is commonly not the case for biological data. Therefore, we did not consider transformers in the thesis [27].

2.2.1 Neural networks

In recent years, the most prevalent type of machine learning algorithms are neural networks. The smallest neural network can be considered a logistic regression [28], where the output value is a linear combination of the input features with an additional bias term and a sigmoid transformation. Neural networks gain their modeling power from stacking these transformations in layers and adding other non-linear activation functions between them. Different compositions of neurons into layers have specific names (e.g., convolutional, recurrent), while the composition of layers and connections between them are called architectures. Some architectures are better suited for solving specific problems; therefore, choosing the model architecture is an important step in modeling the data [29].

2.2.2 Autoencoders

Autoencoders are neural networks with the encoder and decoder components [30]. The encoder component maps a high-dimensional input space into a low-dimensional latent space by discovering patterns in the data. On the contrary, the decoder maps the latent space back into a high-dimensional space (see Fig. 2.2). Mapping to low dimensional space allows the model to only retain crucial patterns while discarding redundant information.

The most common approach to training autoencoders is to minimize the mean squared loss between the input vector x and the reconstructed vector $\hat{x}$ (see Eqn. 2.2):

$$\mathrm{MSE}(x, \hat{x}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2 = \frac{1}{n}\|x - \hat{x}\|_2^2. \tag{2.2}$$

Importantly, the type of loss function determines the meaning of the decoder output vectors. At the same time, reducing the number of latent space dimensions is crucial for the autoencoder to learn non-trivial mappings. For example, the latent space with the same dimension as the input space could

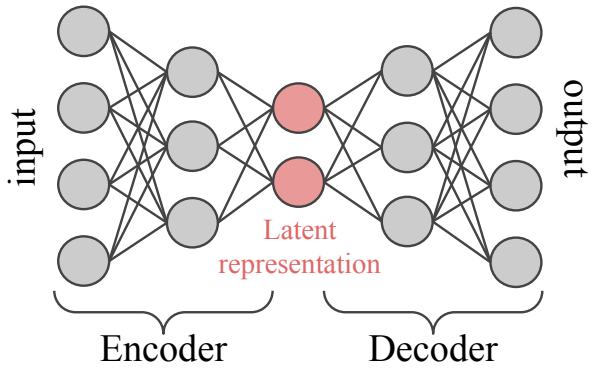

Figure 2.2: Example architecture of an autoencoder. High-dimensional input space is encoded into a lower dimensional latent space via the encoder. Latent space is decoded back into the original input space as a reconstruction. The error between the input values and the output reconstructions is minimized to updated model weights. Lower dimension of latent spaces ensures regularization of the training process.

learn a trivial one-to-one mapping between the input and reconstructed features [30].

2.2.3 Variational Autoencoders

Variational autoencoders (VAEs) extend traditional autoencoders by introducing probability into the latent space. Instead of encoding inputs into a deterministic latent vector, VAEs treat the latent variables as random and force the latent distribution to approximate a Gaussian distribution. The concentration of the latent space to a multivariate Gaussian distribution allows the creation of smooth interpolations in the latent space. The decoder part of the autoencoder samples from the latent distribution to reconstruct the input vector as in the traditional autoencoder. The variational approach allows us to generate new samples from the learned distribution and create synthetic samples [17].

Reparameterization Trick

Sampling from the latent distribution is an integral part of VAE inference. However, the sampling process in the latent space disrupts gradient-based optimization. VAEs employ the *reparameterization trick* to enable back-propagation through the bottleneck layer. Instead of sampling $z \sim q(z|x)$, we reparameterize z as:

$$z = \mu(x) + \sigma(x) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1) \tag{2.3}$$

where $\mu(x)$ and $\sigma(x)$ are learned parameters and isolated ϵ is stochastic. This transformation allows gradients to flow through the encoder network, enabling efficient optimization. From a Bayesian perspective, the reparameterization trick helps approximate the true posterior by constructing a continuous latent space that is differentiable.

Evidence Lower Bound (ELBO)

We want the VAE to generate data such that it matches the real data in distribution. The probability $p(x)$ is called the likelihood and our goal is to maximize it. Directly calculating the likelihood or loglikelihood ($logp(x)$) is intractable due to integration over all possible values of the latent space.

Solving the optimization problem involves using variational inference, which allows for an easier approximation of $p(x)$. Instead of directly maximizing the $p(x)$, we can maximize the Evidence Lower Bound (ELBO), defined as:

$$\log p(x) \geq \mathbb{E}_{q(z|x)} \left[\log p(x|z)\right] - D_{KL}(q(z|x) \parallel p(z)). \tag{2.4}$$

The first term $\mathbb{E}_{q(z|x)} \left[\log p(x|z)\right]$, referred to as a reconstruction term, is the expected log-likelihood that measures how well does the reconstructed data match the input data. Assuming that prediction errors are normally distributed around the true values, maximizing the log-likelihood is the same as minimizing the mean-squared error loss. The second term $D_{KL}(q(z|x) \parallel p(z))$ is the Kulbeck-Lieber divergence [31] between the latent variable distribution

$q(z|x)$ and a prior distribution $p(z)$. We want to minimize the divergence to force the $q(z|x)$ to be close to the prior, often chosen as the standard Gaussian. Such prior is acting as a regularization term, resembling the L2 regularization, while the Laplacian prior would resemble the L1 regularization.

Advancements of VAEs

Since the introduction of variational autoencoders in 2013 [17], numerous improvements have been proposed. Improvements mainly focus on the aspect of disentanglement of the latent space. In the original formulation, the reconstruction and KL divergence terms in the ELBO are treated equally. β-VAE [6] introduces a tunable parameter β that controls the balance between the terms. The objective function for Beta-VAEs is given by:

$$\mathcal{L} = \mathbb{E}_{q(z|x)} \left[\log p(x|z)\right] - \beta \, D_{KL}(q(z|x) \parallel p(z)). \tag{2.5}$$

By adjusting the β parameter, the models allow more interpretability and control over the information capacity of the latent space, improving disentanglement.

2.3 Survival Analysis

Survival analysis is the field of statistics studying time-to-event scenarios. The event of interest can be death, the relapse of an addict, the appearance of malignant tissue, the defaulting of a loan recipient, or the malfunction of a car. Survival analysis approaches proved to be applicable not only in the biomedical field but also in marketing, economics, sociology, insurance industry, engineering, and criminology. Before the introduction of survival analysis, the modeling was focused on predicting the event occurrence or mean lifetime. However, when modeling survival data, neglecting to abide by the rules of survival analysis leads to a biased interpretation [32, 33, 34, 35].

2.3.1 Time-to-Event data

Let T be a continuous non-negative random variable denoting the time until the event occurs. The probability density function $f(t)$ denotes the probability of an event occurring at exactly time t, while the cumulative density function $F(t)$ denotes the probability of the event occurring up until the time t. In survival analysis, we often ask questions regarding the event not occurring until time t, therefore, we will be defining functions more suitable to answer such questions, namely the hazard rate $h(t)$, cumulative hazard $H(t)$, and the survival function $S(t)$.

Hazard rate

The hazard rate $h(t)$ is a measure of risk and is defined as the probability of an event occurring if it has not yet occurred up until that point. The measure is defined as

$$h(t) = \lim_{\Delta t \to 0} \frac{P(t \leq T < t + \Delta t | T > t)}{\Delta t}, \tag{2.6}$$

where T is the true time of the event and Δt is a small interval around time T. Leading from the definition, the hazard rate is not a probability measure. The $h(t)\Delta t$ can be interpreted as the conditional probability of the event occurring in the interval $[t, t + \Delta t)$.

The hazard function is most suitable for studying the mechanism of failure. With the only restriction of being non-negative $h(t) \geq 0$, the shape of the curve can take many forms, indicating how the risk of failure changes over time. An increasing hazard rate is typical when observing wear in mechanical or aging in biological systems. A decreasing hazard rate is often associated with transplants where the risk of rejecting the donated organ diminishes over time. A human life can be associated with a bathtub-shaped hazard with higher infant mortality in early and age-related diseases in later stages of life.

Survival function

The survival function $S(t)$ is the most widely visualized and interpreted function in survival analysis. It denotes the probability of the event not occurring up until the time t. Therefore, it can be defined in relation to the cumulative density function of T as

$$S(t) = P(T \geq t) = 1 - F(t) = \int_t^\infty f(x)dx. \tag{2.7}$$

The survival function is a monotone, non-increasing function and shows the expected properties, such as $S(0) = 1$ and $S(\infty) = 0$. The hazard rate is more informative for determining the mechanism of event occurrence, while survival curves are better visual representations of overall life expectancy and better for comparing different mortality rates.

Applying conditional probability to the definition of the hazard rate connects it to the survival function as

$$h(t) = \frac{f(t)}{S(t)}. \tag{2.8}$$

Equivalently, the connection between the two can be derived from Eqn. 2.7 as

$$h(t) = -\frac{d}{dt} log S(t) \tag{2.9}$$

Cumulative hazard

The cumulative hazard function is a cumulative function over the hazard rate denoted as an integral over the hazard rate from 0 to time t. Rewriting the Eqn. 2.9 leads to

$$S(t) = exp\Big(-\int_0^t h(x)dx\Big) = exp(-H(t)). \tag{2.10}$$

2.3.2 Censoring

The occurrence and inclusion of censored data are unique to survival analysis. Censoring occurs when the event is not observed during a pre-specified time

interval. There are three types of censoring: right censoring, where all we know is that the subject is still alive at time t; left censoring, where all we know is that the event occurred prior to a pre-specified interval; and interval censoring, where all we know is that the event occurred during some interval of time (see Fig. 2.3). Most commonly observed and researched are the right-censored data, and most survival analysis tools are suited to address such challenges. We distinguish between two types of right-censored studies. Type 1 censoring is present where the events can be only observed prior to the closing of the study. On the other hand, Type 2 censoring is involved when studies continue until the first N events occur, which is common in testing component lifetime [36]. Our study will focus on the Type 1 right censoring scenario.

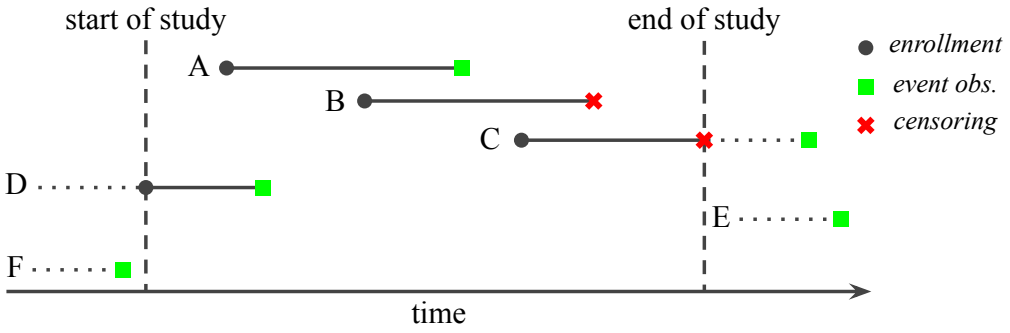

Figure 2.3: Examples of event censoring in survival analysis of Type 1. Only enrolled samples can be included in the study regardless of event censoring, i.e., samples A, B, C, and D. Sample A is an enrolled sample with an observed event. Sample B has censored the event due to not observing the event and missing follow-up. Sample C observed event after the end of the study, therefore, it is censored for the purpose of the study. Samples B and C are right censored. Sample D is left censored. Samples E and F are not enrolled, and their events occur outside of the study period, they are not included in the study.

In the data-gathering phase of our study, we can only observe events that occur while the study is being conducted. Censoring relates to when

the observed event occurs outside of our observation interval or factors non-related to the study caused the event. For example, in a cancer-drug clinical trial, if a patient died in 2024 but we only gathered the data until the end of 2023, their event will be censored. Likewise, if they died in a car accident in March 2023, their event will be censored, since the cause of death is not cancer-related.

2.3.3 Modeling survival data

Estimating the hazard rate or survival function requires modeling of survival data and handling of censored observations. The field offers non-parametric, parametric, and semi-parametric estimators of both while making assumptions about the underlying data-generating process.

Non-parameteric models

The most widely used non-parametric model of the survival function is the Kaplain-Meier estimator [37]. The term *non-parameteric* means that the model does not make any distributional assumptions about the survival curve or the hazard but estimates them directly from the data. The only assumptions of the Kaplain-Meier estimator are uninformative censoring and independence of samples in the study regardless of their enrollment time. Making fewer assumptions than parametric models makes the KM estimator more reliant on large samples to estimate the survival curve (see Fig. 2.4). The resulting curve is a piecewise constant empirical survival curve.

In the clinical setting, one of the most important single-value summaries of the survival curve is the median survival time. It depicts the time at which the half of observed subjects in the study were still alive. On the other hand, for comparing survival curves between different treatment strategies, the 1-year and 5-year survival probability are most commonly compared.

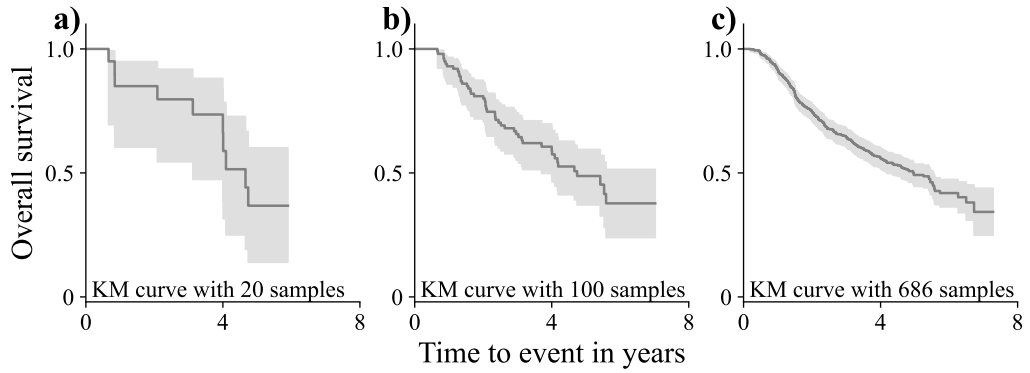

Figure 2.4: Example Kaplain-Meier curves for German Breast Cancer Study Group. Curves are calculated from a random batch of **a**) 20, **b**) 100 and **c**) 686 samples or a full dataset [38]. Shaded region represents 95% confidence interval that decreases with the number of samples in the calculation.

Constructing a likelihood

The prerequisite for parametric modeling of survival curves is a framework for constructing a likelihood. In constructing the likelihood, we assume that censoring is independent during the duration of the study (independent censoring assumption) and that censoring carries no information about the individual status (non-informative censoring assumption). Ensuring the validity of these assumptions caution is required during the study design. The following derivation is inspired by derivation in [39] and [40].

Let t_i denote the survival time of the i-th individual out of n in the study. Let δ_i be the study event indicator, where $\delta_i = 1$ represents occurrence and $\delta_i = 0$ censoring of the event. Let the T_i and C_i denote the time until the event occurs or the event is censored, respectively. Since we can only observe the shorter of both T_i and C_i, we construct $\tau_i = \min(T_i, C_i)$.

We can consider the case when $\delta_i = 0$:

$$P(\tau_i = t, \delta_i = 0) = P(C_i = t, T_i > t) \overset{\text{indep. cens.}}{=} P(C_i = t)P(T_i > t). \tag{2.11}$$

Considering $P(C_i = t)$ is the probability density function of C_i, we get $P(\tau_i =$

$t, \delta_i = 0) = f_{C_i}(t)S_{T_i}(t)$, where $S_{T_i}(t)$ is the survival function of T_i.

We can now consider the reversed case when $\delta_i = 1$:

$$P(\tau_i = t, \delta_i = 1) = P(T_i = t, C_i > t) \overset{\text{indep. cens.}}{=} P(T_i = t)P(C_i > t). \tag{2.12}$$

We can again simplify and say that $P(T_i = t) = f_{T_i}(t)$, which results in $P(\tau_i = t, \delta_i = 1) = f_{T_i}(t)S_{C_i}(t)$, where $S_{C_i}(t)$ is the survival function of C_i.

The likelihood is a product over all n individuals in the study resulting in:

$$\mathcal{L} = \prod_{i=1}^{n} \left[f_{T_i}(t_i)S_{C_i}(t_i)\right]^{\delta_i} \left[f_{C_i}(t_i)S_{T_i}(t_i)\right]^{(1-\delta_i)}, \tag{2.13}$$

which can be reordered by the event indicator as:

$$\mathcal{L} = \prod_{i=1}^{n} f_{T_i}(t_i)^{\delta_i} S_{T_i}(t_i)^{(1-\delta_i)} \prod_{i=1}^{n} f_{C_i}(t_i)^{(1-\delta_i)} S_{C_i}(t_i)^{\delta_i}. \tag{2.14}$$

The second term in the Eqn. 2.14 is considered constant under the non-informative censoring assumption, therefore, we can omit it in construction of the likelihood. The likelihood can be formulated as:

$$\mathcal{L} = \prod_{i=1}^{n} f_{T_i}(t_i)^{\delta_i} S_{T_i}(t_i)^{(1-\delta_i)}. \tag{2.15}$$

Parametric models

Parametric models optimize the parameter choice of the predefined functions, such that they maximize the constructed likelihood. To specify the expected behavior of our survival problem, we have to make an assumption about either the hazard rate or the survival function in the likelihood (Eqn. 2.15) since they cannot be selected independently. Usually, we opt to make the assumption about the hazard and derive the appropriate survival function accordingly.

The simplest choice of hazard rate is a constant hazard $h(t) = \lambda$. From Eqn. 2.10, we can construct an exponential survival curve as

$$S(t) = exp\Big(-\int_0^t h(x)dx\Big) = exp\Big(-\lambda \int_0^t dx\Big) = exp(-\lambda t), \tag{2.16}$$

therefore, the model is called an exponential survival model. The likelihood has a closed-form solution from its only parameter $\lambda = \frac{d}{\sum_i^n t_i}$, with d denoting the number of observed events.

In reality, we rarely expect that hazard rate to be constant in time. Therefore, we introduce more complex hazard functions to better describe the underlying process. Unlike the exponential survival model, none of the other models have a closed-form solution. Thus, numerical methods have to be used to optimize model parameters.

Model	**Hazard** $h(t)$	**Survival F.** $S(t)$	**Parameters**
Exponential	λ	$\exp(-\lambda t)$	$\lambda > 0$
Weibull	$\lambda\gamma t^{\gamma-1}$	$\exp(-\lambda t^{\gamma})$	$\lambda > 0, \gamma > 0$
Log-logistic	$\frac{\lambda\kappa(\lambda t)^{\kappa-1}}{1+(\lambda t)^{\kappa}}$	$\frac{1}{1+(\lambda t)^{\kappa}}$	$\lambda > 0, \kappa > 0$

Table 2.1: Common examples of survival functions.

Survival time and event occurrence are often accompanied by other features describing a subject. These features can be included in modeling as covariates x_i by constructing a linear estimator βx and calculating survival function parameters as a transformation of the product $\lambda = f(\beta x)$. For example, adding covariates to the exponential model means reformulating $\lambda = e^{\beta x}$, since λ cannot take negative values. Consecutively, maximizing likelihood $L(t;\lambda)$ becomes maximizing $L(t;\beta,x)$. The interpretation of model parameters β becomes dependent on the meaning and the transformation of underlying model parameters. However, generally, they relate to mean survival time where higher β values relate to longer survival time.

2.3.4 Proportial Hazards Models

Specifying the hazard function prior to modeling can allow modeling with less data and reduce uncertainty in results. However, assumptions about the hazard can also be restrictive if the assumption is not correct. It is uncommon to know the shape of the hazard without observing the data.

Therefore, models were introduced to avoid making assumptions about the shape of the hazard, making them suited for modeling a variety of survival data.

The proportional hazards assumption indicates that the hazard rate is proportional between samples. Proportional hazard assumption thus assumes that covariates influence the hazard as a multiplication factor to the function and the influence is constant over time. Thus, for every sample, the model can calculate a continuous value representing this multiplicative factor called the risk. With the increasing risk factor, the probability of the event increases, therefore, the survival curve rapidly decreases over time.

Cox Proportional Hazards Model

Cox proposed a semi-parametric proportional hazards model (i.e., CoxPH) combining both concepts outlined above [41]. The model defines the hazard function $h(t; x)$ as

$$h(t; x) = h_0(t)e^{\beta x}, \tag{2.17}$$

where $h(t)$ represents a baseline hazard function, and $e^{\beta x}$ represents the multiplicative factor defined as the product of model parameters β and the sample-specific covariates x. Linear regressor βx can output positive and negative values alike, however, $e^{\beta x}$ maps the values to positive values, satisfying the conditions of proportional hazards. From definition, baseline hazard would still need to be defined, however the models likelihood avoids using it in the calculation by focusing on the proportionality of sample hazards.

CoxPH partial likelihood

CoxPH avoids using baseline hazard in the likelihood function by constructing the likelihood as the ratio between hazards. As depicted in the Eqn. 2.18, the common term $h_0(t)$ is subtracted from the fraction.

$$\frac{h(t; x_1)}{h(t; x_2)} = \frac{\cancel{h_0(t)}e^{\beta x_1}}{\cancel{h_0(t)}e^{\beta x_2}} = e^{\beta(x_1 - x_2)} \tag{2.18}$$

To arrive at the likelihood equation, we have to define the risk set. Let $t_1 < t_2 < ... < t_n$ be ordered survival times of subjects with no ties. The risk set at time $R(t_i)$ is the set of all samples that are still in the study and have not experienced the event. Thus, those subjects with survival times $t_j \geq t_i$. Note that the sample with survival time t_i is included in the risk set $R(t_i)$.

The following likelihood does not consider the baseline hazard distribution and the distribution of censored events and is thus considered as partial likelihood and defined as

$$L(\beta) = \prod_{i=1}^{K} \frac{e^{\beta x_i}}{\sum_{t_j \geq t_i} e^{\beta x_j}}. \tag{2.19}$$

The outer product goes across all K samples with an observed event, whereas the censored samples are not considered. The inner part is the ratio between the risk of the i-th sample and the summed risk of all samples in the risk set at time t_i. Censored samples with survival time beyond t_i are considered in the risk set. Note that the absolute values of survival times are not considered, just their ordering.

The intuition behind the CoxPH likelihood is that the risk of samples with shorter survival times should be higher than the risk of samples with longer survival times. Not considering censored samples follows this intuition since we cannot definitively determine the order of samples and the risk set when the event occurs. The model parameters β relate to risk and not survival time, therefore, their meaning changes in comparison to parametric models. Higher β values relate to higher risk and lower survival time [41].

2.3.5 Deep learning survival models

Since linear estimators of model parameters can only capture linear relationships between covariates, researchers proposed replacing the linear with more complex estimators (e.g., neural networks). The first attempts were made in the late 90s, where only shallow neural networks were used [42, 43, 44]. More recently, researchers introduced improved and deeper models in the form of

DeepSurv [45], DeepHit [46], SurvNet [47] and others. They all estimate the risk for each sample and then calculate the CoxPH likelihood for the entire set. The negative log-likelihood formulation is used to optimize the model parameters.

Importantly, since CoxPH partial likelihood requires the risk of all samples, the loss cannot be calculated for each mini-batch of samples. Current approaches propose calculating the risk for all samples and then calculating the loss to propagate back cumulative gradients for all samples. A recent paper on large-scale training of survival models showed that generating batches of samples with random sampling with replacement can produce unbiased estimates of CoxPH likelihood for smaller batch size [48]. This approach to generating batches can increase the efficiency of training models on large datasets where accumulating gradients can be limited.

Chapter 3

Decomposition of transcription and mRNA decay

To understand the proposed approach, we need to introduce the basics of mRNA dynamics and the fundamentals behind modeling these dynamics using variational autoencoders. The standard approach to modeling mRNA expression data is to reconstruct the input expressions on the output of the autoencoder. The mean-squared error between the input and the reconstructed values is used in training (see Fig. 3.1a,b).

In section 3.1, we introduce the expression data and differential equations for mRNA dynamics involving transcription and mRNA decay. We derive a simple loss function by explicitly dividing transcription and decay rates (i.e., DIV, see Fig. 3.1c). In the following section 3.2, we introduce the fundamentals of survival analysis and transform mRNA dynamics into a survival problem. To tackle the indifferentiability of the CoxPH partial likelihood, we describe constructing time-differentiable CoxPH partial likelihood in section 3.3. Lastly, in sections 3.4 and 3.5 we introduce our transcription-decay decomposition loss function (i.e., TDD, see Fig. 3.1b) and describe the implementation details. To summarize, instead of minimizing the reconstruction loss, we introduce mRNA dynamics constraints as survival-related loss functions.

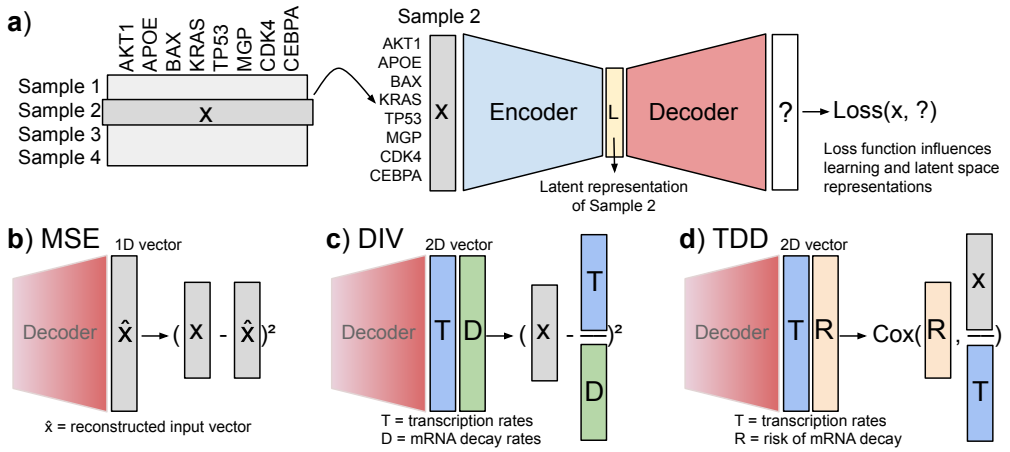

Figure 3.1: Schema of autoencoder training using different loss functions. The choice of the loss function influences the learning of the latent space. **a**) Measurements of mRNA represent input features for each sample. Samples are passed individually into the encoder to learn latent representations. The decoder maps latent space to a high-dimensional space according to the loss function. The meaning of the decoder output is based on the choice of the loss. **b**) The decoder output represents a reconstructed input space vector when using means squared error loss. **c**) Decoding two vectors as transcription and mRNA decay rates allow us to incorporate first-order mRNA dynamics principles. **d**) Using Cox proportional hazard loss with two vector outputs relates to transcription rates and mRNA decay risk.

3.1 First-order dynamics of mRNA concentration

The genetic material in the cell is the static manual on how different cells should function. However, the specific expression and, consequently, concentration of mRNA determine the current state and phenotype of the tissue. Quantification of mRNA molecules in the tissue produces an mRNA expression matrix (see Table 3.1), where each matrix entry represents the estimated number of mRNA molecules transcribed from a given gene. During this pro-

cess, every mRNA from the target tissue is sequenced at once, so the resulting counts reflect the relative concentration of mRNA. Adjusting for both the length of each mRNA molecule and the total number of molecules captured, these values are normalized and presented as transcripts per million (TPM). Since values from samples are normalized, we can compare the concentration of the same mRNA across samples.

Table 3.1: Example RNA-Seq expression counts matrix for selected human genes across different samples. Rows represent samples (e.g., cancer patients), and columns represent how many mRNA molecules of the specific gene were measured in the sample. These values represent mRNA counts related to mRNA concentration in the sample (i.e., $[mRNA]$). The matrix usually consists of more than twenty thousand genes (columns).

Sample	TP53	BRCA1	...	VEGFA	EGFR
Sample 1	10	200	...	3	50
Sample 2	50	195	...	24	70
Sample 3	200	180	...	130	100
Sample 4	0	175	...	9	1500

Let $[mRNA]$ be the number of mRNA molecules in a sample (i.e., concentration). Concentration at any moment depends on synthesis (i.e., transcription) and mRNA decay. We write the concentration dynamics in time t as a function of transcription rate (γ_t) and decay rate (γ_d) as defined with Eqn. 3.1 (from [49]).

$$\Delta[mRNA] = \gamma_t \Delta t - \gamma_d [mRNA] \Delta t \tag{3.1}$$

Given the homeostatic nature of cells and the large amount of sequenced cells, we can simplify the equation and assume a steady-state concentration of mRNA in a given sample. This assumption is equivalent to setting a partial derivative of $[mRNA]$ over time to zero. Thus, we can express the

steady-state concentration of mRNA in terms of transcription and mRNA decay rates (see Eqn. 3.2):

$$\frac{\partial[mRNA]}{\partial t} = \gamma_t - \gamma_d[mRNA] = 0 \Rightarrow [mRNA] = \frac{\gamma_t}{\gamma_d}. \tag{3.2}$$

We propose calculating transcription and decay rates independently and then performing the division. Such modeling of mRNA concentration results in the explicitly divided transcription-decay rates (DIV) loss (see Eqn. 3.3), which has the same assumption as MSE but incorporates the constraint in the model architecture.

$$DIV(x_{input}, \gamma_t, \gamma_d) = MSE(x_{input}, \frac{\gamma_t}{\gamma_d}) = \frac{1}{n}\sum^{n}(x_{input} - \frac{\gamma_t}{\gamma_d})^2 \tag{3.3}$$

3.2 Modeling mRNA concentration with survival analysis

Survival analysis is a branch of statistics handling time-to-event data, such as time from transcription to decay. For example, transcription generates a new mRNA molecule which starts the mRNA lifecycle, while the decay event occurs after some time t, ending the cycle. Apart from modeling mRNA decay as a function rather than a constant, the benefit of survival analysis compared to regression modeling is handling missing event data (i.e., censored data). Inefficient detection of mRNA molecules or measurement errors can be considered censored events.

We propose rearranging the Eqn. 3.2, leading to the survival time proxy of mRNA ($\hat{t}$) as the mean lifetime of the mRNA (see Eqn. 3.4). The survival time proxy is the expected time between transcription and decay of specific mRNA molecules. Interpreting the decay rate in terms of proxy survival time enables us to use survival analysis methods to model the rate.

$$[mRNA] = \frac{\gamma_t}{\gamma_d} \Rightarrow \frac{1}{\gamma_d} = \frac{[mRNA]}{\gamma_t} = \hat{t} \tag{3.4}$$

We can relate this interpretation of the steady-state equation to modeling reconstruction with the MSE and the DIV loss function. Modeling the mean survival time using these functions implicitly assumes the exponential survival model. Equivalently, these functions assume a constant hazard function over time ($h(t) = const.$), meaning that the likelihood of mRNA molecule decaying is constant. Relating to the survival analysis equivalent, modeling survival time with a single parameter λ is the same as fitting an exponential survival model with the mean $\mu = \frac{1}{\lambda} = \hat{t}$. Therefore, modeling the mean survival time allows only a change in the hazard's value but not the curve's shape. Following the mRNA concentration dynamics, modeling the target concentration becomes the product of transcription rate and survival proxy time (i.e., $[mRNA] = \gamma_t \hat{t}$). Since this is equivalent to using the DIV loss function, it makes the same underlying constant hazard assumption.

3.3 Time-differentiable CoxPH partial likelihood

A constant hazard assumption is often violated when considering different mRNA molecules and the regulation of decay. Therefore, we want to loosen the constant hazard assumptions by making the proportional hazard assumption using the Cox proportional hazard model. CoxPH is a semi-parametric model that constructs a partial likelihood as a ratio between samples.

The equation of the CoxPH partial likelihood calculates the product of ratios between i-th samples' risk and the cumulative risk of all samples still present at survival time t_i (see Eqn. 3.5).

$$L(\beta) = \prod_{j=1}^{K} \frac{e^{\beta x_j}}{\sum_{t_i \geq t_j} e^{\beta x_i}} \tag{3.5}$$

Survival time in the above equation is only used for ordering samples in increasing order. Given the i-th sample and its survival time t_i, we can increase or decrease t_i for a small δt such that the order of samples according

to time does not change. The CoxPH partial likelihood does not change as a result. Thus, the partial likelihood derivative w.r.t. survival time is stepwise constant.

To model survival time proxy efficiently with autoencoders, the partial derivative has to have a non-zero derivative almost everywhere. Therefore, we borrow the CoxPH partial likelihood and propose changes in formulation. We aim to replace the discrete survival time t_i of the sample with a Gaussian distribution over the survival time:

$$t_i \sim N(\mu_{t_i}, \sigma^2), \tag{3.6}$$

where σ^2 is a predefined variance.

Introduced formulation changes the definition of the risk set. The risk set is not a fixed set of individuals but a set of all samples with some probability $P(t_i \geq t_j)$. The likelihood also becomes a function of survival times $T = [t_1, ..., t_K]^T$ as defined in Eqn. 3.7:

$$L(\beta, T) = \prod_{j=1}^{K} \frac{e^{\beta x_j}}{\sum_i P(t_i \geq t_j) e^{\beta x_i}}. \tag{3.7}$$

The modified partial likelihood $L(\beta, T)$ does not rely on the survival time distribution being Gaussian. We propose assuming a Gaussian distribution over survival time to simplify the following calculation and training time. One could also consider using β distribution by bounding relative survival time. Assuming a Gaussian distribution over survival times with some variance σ^2 allows for the probability $P(t_i \geq t_j)$ to equal the difference of two i.i.d. Gaussian distributions (see Eqn. 3.8)

$$P(t_i \geq t_j) = P(t_i - t_j \geq 0) = 1 - \phi_{ij}(0), \tag{3.8}$$

where $\phi(0)_{ij}$ is the cumulative density function of $N(t_i - t_j, 2\sigma^2)$ evaluated at zero. Note that $P(t_i \geq t_j) = 1$, where $i = j$, because the sample itself is always in its own risk set. Due to the survival time proxy estimation from the data, there is no event censoring by default.

The extended formulation can still be used to model survival data and matches the CoxPH partial likelihood exactly for the small enough σ^2 with unique survival times for each sample. In addition, it also handles tie-breaks out of the box. With relevance to our work, extending the CoxPH partial likelihood allows us to use it in a self-supervised setting, where automatic derivation can be done for both survival time and risk coefficients.

3.4 Transcription-decay decomposition loss

We propose using the time-distributed CoxPH loss function to model mRNA concentration dynamics in a self-supervised setting. We leverage autoencoders with a slight variation on the last layer, to predict two values for each input instead of one. These two values correspond to the mRNA decay risk ($R_i = e^{\beta X_i}$) and transcription rate ($\gamma_{t,i}$). Dividing the input feature value ($[mRNA]_i$) with the predicted transcription rate results in the survival time proxy ($t_i = \frac{[mRNA]_i}{\gamma_{t,i}}$, see Eqn. 3.4). The procedure can be calculated for all output features simultaneously.

Both mRNA decay risk and the survival time proxy are used to calculate the likelihood of time-distributed CoxPH partial likelihood. Eqn. 3.9 represents the likelihood formulation:

$$L(x_{input}, T, R) = \prod_{j=1}^{K} \frac{R_j}{\sum_i P(t_i \geq t_j) R_i}, \tag{3.9}$$

where x_{input} are input values, and vectors γ_t and R correspond to the transcription rate and mRNA decay risk outputs of the VAE. The time variable t_i is calculated by diving input value x_i with predicted transcription rates $\gamma_{t,i}$.

The proposed transcription-decay decomposition loss function is a negative log-likelihood of Eqn. 3.9 denoted as (see Eqn. 3.10):

$$TDD(x_{input}, T, R) = -\frac{1}{K} \sum_{j=1}^{K} \left(\log(R_j) - \log \sum_i P(t_i \geq t_j) R_i \right). \tag{3.10}$$

3.5 Implementation details

Here, we describe the implementation of the loss function in the *pytorch* library [50], computational bottlenecks with regards to the TDD loss, and the construction of an orthogonal penalty to the latent space dimensions.

3.5.1 Adjusting TDD to gene expression data

Expression values from RNA-seq experiments are usually log-transformed using pseudo 1 count. Therefore, the input expression values x_{input} are interpreted as $\log([mRNA])$. The transcription rate should be positive, thus we predict values as $\log(\gamma_t)$. The corresponding survival time proxy is calculated as:

$$\hat{t} = exp\left(\log([mRNA]) - \log(\gamma_t)\right). \tag{3.11}$$

Since absolute values of survival time are not considered, the survival time is to the $[0, 1]$ interval for numeric stability of other parameters.

The implementation of the TDD in PyTorch follows a four step process 3.1. The input variables are tensors of the shape n for samples and m for genes. *Log_risk* and *transcription* are predicted by the autoencoder, *expressions* are the input features, *event* is a matrix of ones, and variance is the hyperparameter for the time distribution. In the first step (lines 10-13), we calculate the survival time proxy, normalize the values between $[0, 1]$, and divide by the $2\sigma^2\sqrt{(2)}$ to normalize time location to unit variance. The second step (lines 15-17) calculate the pairwise difference in proxy survival time as the location parameter of the Gaussian distribution. Note that the calculation is done using view of the tensors, since the *location* tensor is of shape (n, n, m). In the line 16, we calculate the *phi_matrix* as the CDF of a Gaussian distribution with unit variance evaluated at 0. The *torch.erf(...)* step is the most computation and memory heavy step, since it requires to store an (n, n, m) matrix in GPU memory, while computing the integration of (see Eqn. 3.12):

$$r_f(x) = \frac{\pi}{2}\int_0^x e^{-t^2}\,dt, \tag{3.12}$$

where x is the location of the Gaussian. In line 17, we add 0.5 to the diagonal values, since evaluation returns 0.5, however, we know that the probability of a sample to be in its own risk set is 1. The third step is finally scaling down the matrix to calculate the cumulative risk based on the probability of inclusion in the risk set *phi*. The variable name *cumsum* is used to indicate the common point between the traditional and time-distributed implementation of the algorithm. The final step in the calculation is calculating the negative log-likelihood from the Eqn. 3.10.

```
1  def forward(self,
2          log_risk,        #torch.Tensor shape (n, m)
3          expressions,     #torch.Tensor shape (n, m)
4          transcription,   #torch.Tensor shape (n, m)
5          event,           #torch.Tensor shape (n, m)
6          variance,        #torch.float
7      ):
8      n, m = expressions.shape #n-samples, m-genes
9
10     time = (expressions - transcription).exp()
11     time -= time_indicator.min(dim=0)[0]
12     time /= time_indicator.max(dim=0)[0]
13     time /= (2 * variance * torch.sqrt(2)))
14
15     location = time.view(n, 1, m) - time.view(1, n, m)
16     phi_matrix = 0.5 * (1 + torch.erf(-location))
17     phi_matrix[torch.arange(n), torch.arange(n)] += 0.5
18
19     risk = log_risk.T.exp().view(m, 1, n)
20     phi = phi_matrix.permute((2,1,0))
21     cumsum = torch.matmul(risk, phi).squeeze()
22
23     log_cumsum = torch.log(cumsum).T
24     return -torch.sum((log_risk - log_cumsum) * event)/n
```

Listing 3.1: TDD implementation in PyTorch

3.5.2 Computational bottleneck in TDD

When calculating the loss with common machine learning loss functions such as MSE, we can calculate the contribution of the i-th sample in isolation from other samples. With CoxPH likelihood, such isolations can only be done up to the calculation of the i-th sample risk. Then, the contribution of the i-th sample to the likelihood depends on the risk of samples in the risk set. As such, the likelihood should preferably be calculated on the whole dataset at a time or at least as large of a batch as possible. The larger the batch, the better the approximation of the data-generating process. In our work, when the dataset size exceeded the limits of the available hardware, we created batches of samples using random sampling as discussed in the literature [48].

The bulk of the calculation is performed to obtain the sum of samples in the risk set (see Listing 3.1, line 16). Evaluating this sum using the standard CoxPH partial likelihood can be simplified using the cumulative sum trick. Since the risk sets are incremental and strict subsets in accordance to survival time, the summed risk can be calculated with a cumulative sum. Using time-distributed CoxPH, we lose this privilege, since samples with shorter survival time are still included in the risk set of samples with longer survival time, making the cumulative sum impractical to calculate. Therefore, to calculate the sum of the i-th sample risk set, we have to evaluate $\phi_{ij}(0)$ for every sample in the dataset or batch. In practice, we calculate an (n, n) matrix of ϕ evaluated at 0 for every sample pair. In addition, this matrix is calculated for every out of m input features, thus creating the (n, n, m) tensor of evaluations. This calculation requires a huge amount of GPU storage to calculate and is the limiting factor when using our approach. The calculation scales linearly with the input features m and quadratically with the size of the batch n. The sequential calculation does not alleviate hardware constraints since the computational graph stores the intermediate results for the purpose of backpropagation.

3.5.3 The σ^2 parameter influences sample shuffling

With the time-distributed CoxPH partial likelihood, we introduced the distribution over survival time (see Eqn. 3.6). The mean parameter is calculated according to Eqn. 3.4, whereas variance σ^2 can be set manually prior to training. In a standard survival analysis study, the assumed distribution of time can be set in accordance with the expected measurement error distribution, with the variance parameter relating to the expected uncertainty. In practice, it might be more meaningful to consider standard error in units of time and later convert it to variance. When considering survival time as an input feature, the variance parameter only influences the likelihood.

When using the TDD loss, the predicted proxy survival is batch normalized to lay on the interval [0, 1], which unifies the parameter choice over multiple batches. Since only the relative positioning of survival times is used in the calculation, we do not discard any information. As opposed to the standard survival problems with fixed survival times, in our case, the gradient also propagates back through the predicted survival time. Consequently, the variance parameter plays a role in adjusting predicted survival time during training. Distributed survival time allows samples with lower survival time to be also included in the risk set of samples with higher survival time. Higher variance values extend the reach of the sample, resulting in it being included in higher survival time risk sets with higher probability. Higher variance also results in smoother likelihood distribution, allowing for incremental optimization of survival times. Therefore, the higher the variance parameter, the more easily survival time is optimized and the relative ordering of samples is changed. However, too high a variance makes survival time and risk sets irrelevant since every sample has roughly the same probability of being in the risk set of another example.

In this work, we used a variance scheduler to adjust the parameter value during training. Variance was incrementally reduced from 10^{-2} to 10^{-8} over the course of 1000 epochs. The starting variance allowed for sufficient reordering, such that the weight initialization did not play a crucial role. Over

the course of the training, the lower bound changed the focus of the TDD loss toward optimizing the risk estimator. Scheduler parameters were not subject to hyperparameter optimization to reduce search space. In future work, we aim to explore this variance tuning and its effects in further detail, which is out of the scope of this work.

3.5.4 Promoting orthogonality of the latent space

When a strong signal is present in the data, autoencoders tend to overfit the signal with multiple neurons in the latent space, reducing the effective dimensions of the latent space. To improve the expressiveness of VAE models, we add the orthogonality term to the VAE loss function in addition to the reconstruction and KL-divergence terms. The idea was previously introduced for non-negative matrix factorization [51]. The orthogonality term penalizes correlated latent dimensions.

Let the latent vector l_i be an m dimensional vector of i-th dimension values for n samples a batch. The orthogonality term is defined as (Eqn. 3.13):

$$O_{ortho} = \sum_{i,j}^{m} (cos\theta_{ij})^2 = \sum_{i,j}^{m} (\frac{l_i \cdot l_j}{||l_i||\ ||l_j||})^2. \tag{3.13}$$

It calculates the cosine similarity between latent dimensions of each training batch and adds the sum of squared values to the loss. We implemented the orthogonality penalty in matrix form to improve computation time. Note that latent vectors are always orthogonal to themselves; thus, the diagonal elements are always equal to one. Since the length of latent vectors does not have an influence on the orthogonality penalty, the number of samples in the batch can be adjusted independently.

The orthogonality penalty multiplied by the orthogonality ratio term is added to the ELBO equation. The orthogonality ratio hyperparameter adjusts the influence of the orthogonality term on the loss. The higher the orthogonality the lower the correlation between the latent features, thus higher effective dimensions.

Note that we are not forcing the latent dimensions to be completely orthogonal. Since we are always minimizing squared cosine similarity on a subset of samples, the orientation of dimensions is always relative. Furthermore, the scale of the orthogonality ratio term is largely dependent on the type and the scale of the loss function. MSE and DIV loss functions produce small loss values, therefore, the ratio term can be relatively small. On the contrary, TDD loss calculates the negative log-likelihood with much larger values, thus requiring the orthogonality ratio to be relatively large.

Chapter 4

Evaluation method

We evaluate the impact of the autoencoder loss function by comparing latent representation performance on three downstream tasks, i.e. use latent representations as input data for further modeling. Our experiments follow a three-step protocol. First, we train a fully connected variational autoencoder with a desired loss function using mRNA expression data in a 5-fold cross-validation with hyperparameter tuning (see Fig. 4.1a). Using different loss functions results in different latent representations. Secondly, we embed samples in latent space using the encoder part of the variational autoencoder. Additionally, we add relevant clinical information, such as drug response, survival outcome, and cancer subtype, to enable supervised learning (see Fig. 4.1b). Lastly, we use the latent representations as a new input to perform patient clustering, drug response prediction, and survival outcome modeling (see Fig. 4.1c). For a wholesome comparison, we add PCA embeddings and original input space to downstream modeling.

4.1 Data

We collected 1408 cell-line samples and their corresponding mRNA expression data from the Cancer Cell Line Encyclopedia (CCLE) [52] with associated drug response from screening experiments [53]. Drug response was not

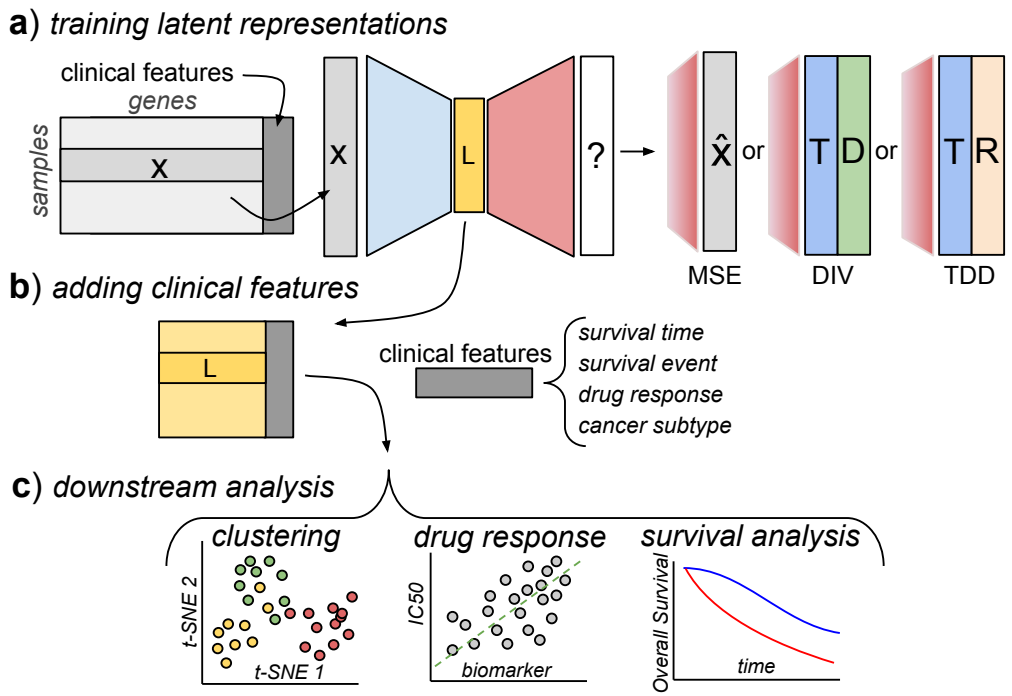

Figure 4.1: Experimental setting for evaluating latent space performance. **a**) Variational autoencoders are trained on mRNA expression data using MSE, DIV, or TDD loss function. **b**) The encoder part transforms gene input space into latent representations. Clinical features are added for downstream analysis. **c**) Evaluation of patient clustering, drug response prediction, and survival outcome modeling.

available for every drug-cell line combination. Additionally, we collected 1980 METABRIC breast cancer patient data [54] with assigned Claudin cancer subtype and progression-free survival outcome. To achieve diversity of cancer types, we also collected samples from The Cancer Genome Atlas (TCGA) [55] relating to five different tissues. We used breast, kidney, and brain cancer projects (TCGA projects BRCA, KIRP, LGG) to evaluate clustering on cancer subtypes as described in [56]. Furthermore, we used kidney, lung, and brain cancer projects (TCGA projects KIRC, LUAD, LGG) to model survival outcomes, as they have the highest number of recorded survival events.

In mRNA expression preprocessing, we first extracted a thousand land-

Table 4.1: Table of datasets used in the evaluation. Only cell-line datasets had measured drug responses available. Clustering was evaluated on clinical datasets if a clear cancer subtypes was described in the literature and already used as a benchmark label. Survival endpoint was used only if there was more than 100 events observed per dataset.

			Downstream tasks		
Dataset	N samples	Type	Clustering	Drug response	Survival
CCLE	1408	celline		✓	
METABRIC	1980	clinical	✓		✓
TCGA-LGG	514	clinical	✓		✓
TCGA-BRCA	1089	clinical	✓		
TCGA-KIRP	283	clinical	✓		
TCGA-KIRC	512	clinical			✓
TCGA-LUAD	567	clinical			✓

mark genes (i.e., L1000 [57]) that incorporate over 80% of the variance in publicly available datasets. Limiting the number of genes to the most highly variable genes or a predefined geneset is a common practice in gene expression modeling to reduce noise. Different sequencing techniques produce slightly different expression distributions, thus we normalize them according to standard protocol. Transcript-per-million expression data was log normalized with pseudo count one and standardized separately across samples for each gene. Microarray expression data already represents log-transformed expression values; thus, only standardization was performed. Loss function implementations are adjusted accordingly to modeling log normalized expression values. Clinical data was not used for training the autoencoders but was added for the evaluation of latent representations (see example in Table 4.2).

Table 4.2: Toy example of RNA-Seq expression counts matrix for selected human genes. There are usually more than twenty thousand genes in the matrix. Each sample has accompanied clinical data, such as cancer subtype (IDH gene mutated or wildtype), IC50 - drug response in terms of inhibitory concentration, PFS - progression-free survival time, and event.

	mRNA expression			Clinical features			
ID	TP53	...	EGFR	Subtype	IC50	PFS Time	PSF Status
1	10	...	50	IDH-mut	14	13	Alive
2	50	...	70	IDH-mut	0.4	43	Dead
3	200	...	100	IDH-mut	5	5	Dead
4	0	...	1500	IDH-wild	2	67	Alive

4.2 Inference of latent representations

We used variational autoencoders (VAEs) with different loss functions to encode the high-dimensional space of genes into latent representation for further downstream analysis. We used MSE, DIV, and TDD losses with VAE while using PCA-embedded and non-embedded input space of L1000 genes to compare downstream task performance. The number of latent dimensionsions was based on the downstream task.

Embedding models were trained in a 5-fold CV with extensive hyper-parameter tuning on a holdout test set taken from the training portion of the data. We used *optuna* library [58] for hyper-parameter tuning with a range of parameters (see Table 4.3). The choice for encoder model architecture was limited to three types: wide (a single layer of 5000 neurons), deep (five consecutive layers of 500 neurons), and cascading architecture (four consecutive layers of 1000, 500, 200, and 100 neurons). The decoder architecture follows the same architecture in reverse order. The number of latent neurons was set to 30 for TCGA datasets, 50 for METABRIC, and 100 for the CCLE

datasets, restricted by the number of samples in the test set in 5-fold CV (such that the number of samples in the test dataset does not exceed the number of latent features as input to downstream models).

Table 4.3: Range of parameter values used in hyperparameter tuning. The scale denotes whether sampling was performed uniformly or logarithmically on the range.

Parameter	*Range of values*	*Scale*
Learning rate	5×10^{-5} – 10^{-3}	Logarithmic
L2 penalty	10^{-3} – 10^{3}	Logarithmic
Orthogonality (β_{ortho})	10^{-3} – 10^{5}	Logarithmic
KLD ratio (β_{KLD})	10^{-4} – 10^{2}	Logarithmic
Hidden layers	wide, deep, cascading	Uniform
Cosine T-max	10 – 200	Uniform

The final variational autoencoder loss function is constructed as:

$$Loss(...) = Loss_{recon.} + \beta_{KLD} D_{KL} + \beta_{ortho} O_{ortho} + \lambda_2 \sum_{i=1}^{n} \beta_i^2, \tag{4.1}$$

where $Loss_{recon.}$ represents either MSE, DIV or TDD loss, D_{KL} represents KL divergence in the VAE, O_{ortho} represents orthogonality penalty term, and the $\sum_{i=1}^{n} \beta_i^2$ represents the L2 penalty of the autoencoder weights. Ratio terms in the loss function are all subject to hyperparameter optimization.

4.3 Evaluation on downstream tasks

To evaluate the added value of mRNA dynamics-inspired loss functions, we compared the performance of latent representations on three of the most commonly used downstream tasks.

4.3.1 Evaluating clustering

We evaluated the clustering performance of latent representations by using Moran's I measure [59]. The measure calculates the spatial autocorrelation between samples. We opted for Moran's I instead of standard cluster-based measures (e.g., Silhouette, Dunn, Davies-Boulding score) because they penalize possible subclusters in the latent space. To calculate the measure, we used t-distributed stochastic neighbor embedding (t-SNE) [60] to further reduce the dimension of our dataset to two dimensions. Then, we constructed a nearest neighbor graph on top of the t-SNE embedded latent space. We weighted the edges according to Gaussian distribution with a mean zero and standard deviation of 5% of the largest distance between two nodes. In essence, the measure relates to the label agreement of samples in the near vicinity. If a larger cluster of samples with the same label is split into two clusters, the autocorrelation stays the same. We used the Claudin subtype for METABRIC, estrogen receptor status for TCGA-BRCA, tumor type for TCGA-KIRP, and IDH1 mutation status for TCGA-LGG samples as clustering information. To quantify the uncertainty in the evaluation, we performed 100 iterations of bootstrap random sampling of samples used to calculate the Moran's I.

4.3.2 Evaluating drug response prediction

To evaluate drug response, we trained the Ridge regressor with 10-fold CV hyperparameter tuning on the training set. Drug responses for 286 drugs with at least 500 responses were used to train the model using MSE loss. Since not all samples had drug responses for all the drugs, the datasets for Ridge regressor training were different. Furthermore, the range of the target variable varies across drugs. Therefore, summarizing regression results across all drugs requires interpreting each drug response as a separate dataset. We transformed latent space performance across methods by ranking the results, where 1 is assigned to the best performing and 5 to the worst-performing

method for each drug. The mean ranks across all drugs were compared with a critical distance diagram [61]. Critical distance was used to evaluate the significance of the results.

4.3.3 Evaluating survival outcome modeling

To evaluate the survival modeling performance of the latent representations, we trained the CoxPH model with an L2 penalty of 0.1 in a 10-fold CV. We used progression-free survival as a survival endpoint. The performance was evaluated on a hold-out test set using the concordance index. Using a higher number of folds is not feasible, as the test set requires a sufficient number of samples. Standard error was calculated over 5-fold CV results.

Chapter 5

Results

By training VAE using different loss functions, we aim to evaluate the benefit of mRNA dynamics-inspired latent representations. We compared the DIV and TDD loss functions in latent space modeling for the three most common downstream tasks: cancer subtype clustering, drug response prediction, and survival outcome modeling.

5.1 TDD improves cancer subtype clustering over MSE

Modeling VAE latent space with MSE and explicitly dividing transcription-decay loss produces latent spaces with similar density and composition, i.e. patient clustering in the t-SNE embedding appears similarly sparse. In contrast, VAE using transcription-decay decomposition loss produces a latent space with more densely clustered samples (see Fig. 5.1). Moran's I measure shows better spatial autocorrelation between samples when using TDD and PCA (see Table 5.1). Higher density in TDD latent space might be the product of the loss function considering multiple samples at once, making it aware of the neighborhood.

In all four clinical datasets, TDD latent representations achieve higher performance compared to the MSE latent representations. The DIV repre-

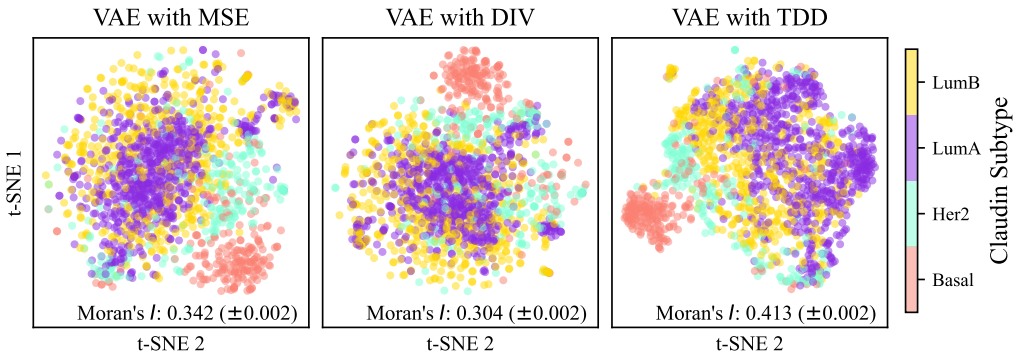

Figure 5.1: Comparison of 50-dimensional latent space embedded with t-SNE based on METABRIC samples. Claudin test estimates subgroups based on biochemical features: Luminal A (rich in estrogen and progesterone receptors), Luminal B (same as A but higher proliferation), Her2 gene overexpressing, and triple-negative Basal cancer.

sentations achieve higher performance than TDD on TCGA-KIRP and lower performance than MSE on METABRIC datasets. Interestingly, DIV and MSE representations consistently underperform compared to the raw L1000 expressions on the four clustering datasets. This would imply that DIV and MSE-extracted patterns are less related to cancer subtypes and less likely to be used as clinical biomarkers. PCA latent representations achieve the highest Moran's I. Due to describing the highest variance with orthogonal latent dimensions, these representations have the highest effective number of

Table 5.1: Latent space performance on clustering. We report the mean Moran's I over t-SNE embedding with bootstrap estimated standard error.

	METABRIC	TCGA-BRCA	TCGA-KIRP	TCGA-LGG
VAE TDD	0.413 ± 0.002	0.596 ± 0.005	0.178 ± 0.006	**0.352** ± 0.008
VAE DIV	0.304 ± 0.002	0.495 ± 0.004	0.204 ± 0.007	0.244 ± 0.011
VAE MSE	0.342 ± 0.002	0.407 ± 0.004	0.144 ± 0.006	0.229 ± 0.010
PCA	**0.462** ± 0.002	**0.614** ± 0.004	**0.325** ± 0.007	0.301 ± 0.011
No Embed.	0.412 ± 0.002	0.569 ± 0.005	0.268 ± 0.007	0.300 ± 0.011

dimensions. As cancer subtypes differ in their mRNA profile due to biological and non-biological effects, high PCA performance is expected.

5.2 TDD achieves the highest performance across drug response predictions

We used latent representations of cell lines to model their response to 286 drugs. The dataset includes a variety of cell lines from different tissues. Thus, the ability of the latent space to differentiate between them is crucial for drug response prediction. The TDD loss achieves the highest mean rank across drug responses with the best performance for roughly three out of four drugs (see Fig. 5.2). The second best-performing method is PCA. Both biology-inspired loss functions outperform MSE, showing the added benefit of explicitly modeling mRNA dynamics. Non-embedded input space consistently underperforms in drug prediction tasks, mostly due to a large number of features compared to a limiting number of training samples. Critical distance (CD) indicates the statistical significance of the results, where results that fall closer than CD are not significantly different.

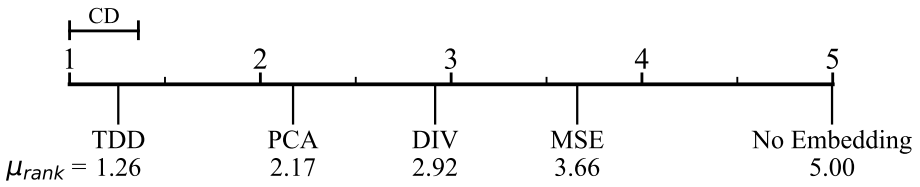

Figure 5.2: Critical difference diagram of method ranks based on drug response data. With 286 drugs used, the critical difference equals 0.361, relating to the minimal distance between ranks still considered significant. Acronyms relate to the method used for latent space modeling: TDD - transcription-decay decomposition loss; DIV - explicitly divided transcription and decay loss; MSE - mean squared error loss.

Cancer drugs are designed to target and act on specific processes within tumor tissue. Some cancer cell lines rely more on the targeted process for survival and proliferation than others. Therefore, latent representations must incorporate both the global tissue type information and local perturbations in the cellular processes to successfully predict response to drugs. PCA optimizes global vectors of the highest variance and achieves solid performance in drug response prediction, mostly due to differentiating cancer types. In this aspect, PCA and TDD representations are similar. However, modeling latent space on batches of samples allows TDD to optimize the local relations and improve differentiation between different drug responses.

5.3 Different latent representations have no significant effect on survival modeling

Drug response in cell lines is dependent on drug-specific processes in relation to cell survival. Modeling survival outcomes adds an additional layer of complexity where these cells reside in biological niches surrounded by healthy tissue and immune cells. Survival analysis in cancer biology aims to discover novel survival-related biomarkers among latent dimensions of expression data. We evaluated how latent representations reflect patients' progression-free survival. In survival analysis, model selection relates to constraints and assumptions about the underlying data. We chose the CoxPH semi-parametric model to avoid making assumptions about the baseline hazard which is also usually done in practice.

Using the concordance index, different latent space representations achieve similar performance across all four clinical datasets (see Table 5.2) with a slight exception of METABRIC. In terms of performance compared to the global representation of the PCA, further optimizing local mRNA dynamics does not seem to be related to survival outcome. CoxPH expects the features to act proportionally to the baseline hazard, rendering local changes insignificant. Therefore, we also observe that non-embedded input space performs

Table 5.2: Evaluating CoxPH with C-index using latent spaces shows the marginal difference between encodings. Progression-free survival was used as a survival endpoint. The standard error is calculated over a 5-fold CV.

Dataset	METABRIC	TCGA-KIRC	TCGA-LUAD	TCGA-LGG
VAE TDD	**0.637** $\pm$ 0.007	<u>0.729</u> $\pm$ 0.028	<u>0.574</u> $\pm$ 0.019	**0.692** $\pm$ 0.016
VAE DIV	<u>0.627</u> $\pm$ 0.006	0.710 $\pm$ 0.021	0.553 $\pm$ 0.018	0.682 $\pm$ 0.014
VAE MSE	0.625 $\pm$ 0.005	0.720 $\pm$ 0.018	0.551 $\pm$ 0.022	<u>0.685</u> $\pm$ 0.014
PCA	0.616 $\pm$ 0.005	0.710 $\pm$ 0.020	0.561 $\pm$ 0.023	0.680 $\pm$ 0.013
No Embed.	0.611 $\pm$ 0.011	**0.731** $\pm$ 0.024	**0.586** $\pm$ 0.011	0.660 $\pm$ 0.010

comparably. The scope of the evaluation is also limited to evaluation on the whole test set as the C-index requires multiple samples to be evaluated.

5.4 Discussion

We have yet to discuss a biological aspect of proposed loss functions. The outputs of the last decoder layer should correspond to some transformation of transcription and decay rates. However, given the stochastic nature of processes occurring on the molecular level, biologists have yet to measure the true rates of these processes. Furthermore, given the numerous factors influencing these rates, we cannot truly compare them in a vacuum. However, we know what is the expected correlation between the transcription and mRNA decay rates. Cellular systems aim to maintain homeostasis, where the cell can quickly react to environmental stimuli. Therefore, small changes in transcription and mRNA decay result in large effects on the concentration of mRNA.

Incorporating biological knowledge into loss functions gives interpretation to the autoencoder outputs. However, loose constraints allow for equivalent solutions and loss of interpretation. We compare transcription and decay rates from DIV and TDD autoencoders with respect to individual genes (see Fig. 5.3a,b,c). Most of the genes exhibit the same patterns, thus we present one of those genes. Observing the training outputs of the autoencoder with

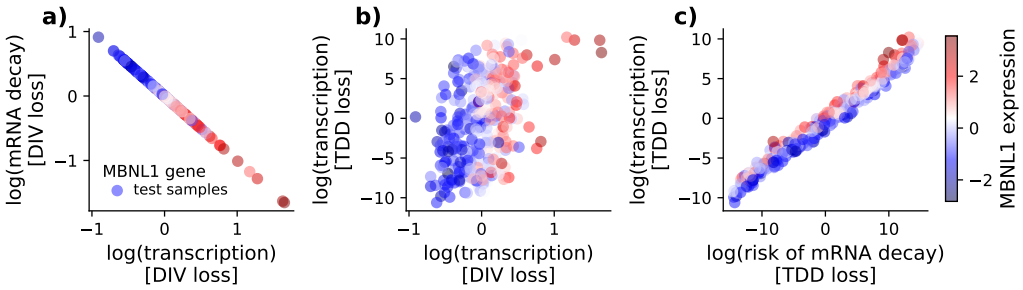

Figure 5.3: Comparison of VAE outputs using mRNA dynamics loss functions. The hyperparameter-optimized model trained on the first CV-fold of METABRIC data was used for showcasing. **a**) Comparing transcription and decay outputs for a single gene using DIV loss, and **c**) TDD loss. **b**) Comparing transcription rates using DIV and TDD losses.

the DIV loss function shows that predicted transcription and decay rates are highly negatively correlated (see Fig. 5.3a). From our knowledge of homeostasis, we would expect the processes to positively correlate. Furthermore, the mRNA expression also correlates with transcription and negatively correlates with mRNA decay, which again violates our background knowledge. The unrealistic interpretation using DIV loss comes from loose constraints on the outputs of the autoencoder, where one of the neurons is redundant. Using DIV loss formulation, there can be multiple equivalent solutions achieving the same loss. However, given that we are interested in the interpretation of the results, most of them are incorrect. On the other hand, correlating the transcription rates shows a low correlation between the two, with DIV rates being more predictive of expression, which agrees with the redundancy of neurons (Fig. 5.3b).

Observing the transcription rate and risk of mRNA predicted by TDD loss autoencoder shows a high correlation between the two predicted values, which is in concordance with homeostasis (Fig. 5.3c). Furthermore, the expression seems to be connected to the difference between log transcription and log risk of mRNA decay which translates to division of transcription and decay. The expression pattern follows the exact relation that we would assume from

our background knowledge and the mRNA dynamics differential equation. Using survival techniques in the form of CoxPH likelihood introduces crucial constraints to the modeling process by considering batches of samples and making assumptions about the underlying hazard. As a result, we are able to correctly model mRNA dynamics with respect to biological regulatory mechanisms.

Chapter 6

Conclusions

Molecular biology is rich in both data and domain knowledge. Data analytics research addressing related problems in biomedicine typically seeks to combine the two in predictive and explanatory models. Domain knowledge serves to guide the modeling process by enforcing additional constraints that allow meaningful representations to emerge. Current research in this field focuses mainly on transforming input data or adjusting the model according to domain knowledge while modifying the loss function remains largely unexplored.

Our work fills the niche by introducing a novel loss function inspired by mRNA dynamics. By acknowledging the two main biological processes governing mRNA concentration, we construct the transcription-decay decomposition loss function used to train variational autoencoders. Resorting to survival analysis techniques allows us to effectively model these processes in accordance with biological expectations. Furthermore, learned sample representations show increased performance on patient clustering and drug response prediction tasks compared to using MSE.

Given unlimited and noiseless data, high-performance modeling can be achieved with any complex machine learning algorithm. However, low sample size and noisy data scenarios call for additional domain expert intervention to direct modeling to meaningful solutions. Our approach aims to bridge this

gap to allow better generalization of results given limited resources, which is mostly the case when modeling clinical patient data.

In this work, we showcased our loss function on a few bulk RNA-Seq expression datasets for the three most common downstream tasks. Alongside extended benchmarking on diverse datasets, we aim to explore the use of TDD on single-cell RNA-Seq expressions where the censoring aspect of the loss can be fully utilized. Furthermore, domain-knowledge-guided loss functions are capable of representing more complex dynamics equations, thus allowing for more comprehensive and meaningful descriptions of the process.

Bibliography

[1] H. Sung, J. Ferlay, R. L. Siegel, M. Laversanne, I. Soerjomataram, A. Jemal, F. Bray, Global cancer statistics 2020: Globocan estimates of incidence and mortality worldwide for 36 cancers in 185 countries, CA: a Cancer Journal for Clinicians 71 (3) (2021) 209–249.

[2] K. Swanson, E. Wu, A. Zhang, A. A. Alizadeh, J. Zou, From patterns to patients: Advances in clinical machine learning for cancer diagnosis, prognosis, and treatment, Cell (2023).

[3] M. Wysocka, O. Wysocki, M. Zufferey, D. Landers, A. Freitas, A systematic review of biologically-informed deep learning models for cancer: fundamental trends for encoding and interpreting oncology data, BMC Bioinformatics 24 (1) (2023) 198.

[4] M. Špendl, T. Curk, B. Zupan, Latent embedding based on a transcription-decay decomposition of mRNA dynamics using self-supervised CoxPH, Springer, 2024.

[5] K. Y. Yeung, W. L. Ruzzo, Principal component analysis for clustering gene expression data, Bioinformatics 17 (9) (2001) 763–774.

[6] D. J. Rezende, S. Mohamed, D. Wierstra, Stochastic backpropagation and approximate inference in deep generative models, in: International Conference on Machine Learning, PMLR, 2014, pp. 1278–1286.

[7] D. Im Im, S. Ahn, R. Memisevic, Y. Bengio, Denoising criterion for variational auto-encoding framework, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 31, 2017.

[8] R. Lopez, J. Regier, M. B. Cole, M. I. Jordan, N. Yosef, Deep generative modeling for single-cell transcriptomics, Nature Methods 15 (12) (2018) 1053–1058.

[9] C. H. Grønbech, M. F. Vording, P. N. Timshel, C. K. Sønderby, T. H. Pers, O. Winther, scVAE: variational auto-encoders for single-cell gene expression data, Bioinformatics 36 (16) (2020) 4415–4422.

[10] J. Ma, M. K. Yu, S. Fong, K. Ono, E. Sage, B. Demchak, R. Sharan, T. Ideker, Using deep learning to model the hierarchical structure and function of a cell, Nature Methods 15 (4) (2018) 290–298.

[11] K. Y. Michael, J. Ma, J. Fisher, J. F. Kreisberg, B. J. Raphael, T. Ideker, Visible machine learning for biomedicine, Cell 173 (7) (2018) 1562–1565.

[12] D. Doncevic, C. Herrmann, Biologically informed variational autoencoders allow predictive modeling of genetic and drug-induced perturbations, Bioinformatics 39 (6) (2023).

[13] L. Seninge, I. Anastopoulos, H. Ding, J. Stuart, VEGA is an interpretable generative model for inferring biological network activity in single-cell transcriptomics, Nature Communications 12 (1) (2021) 5684.

[14] G. O. Consortium, The Gene Ontology (GO) database and informatics resource, Nucleic Acids Research 32 (suppl_1) (2004) D258–D261.

[15] X. Chen, L. Wang, J. D. Smith, B. Zhang, Supervised principal component analysis for gene set enrichment of microarray data with continuous or survival outcomes, Bioinformatics 24 (21) (2008) 2474–2481.

[16] H. Zou, T. Hastie, R. Tibshirani, Sparse principal component analysis, Journal of Computational and Graphical Statistics 15 (2) (2006) 265–286.

[17] D. P. Kingma, M. Welling, Auto-encoding variational bayes, arXiv preprint arXiv:1312.6114 (2013).

[18] R. Xie, J. Wen, A. Quitadamo, J. Cheng, X. Shi, A deep auto-encoder model for gene expression prediction, BMC Genomics 18 (2017) 39–49.

[19] L. Ferraro, G. Scala, L. Cerulo, E. Carosati, M. Ceccarelli, MOViDA: multiomics visible drug activity prediction with a biologically informed neural network model, Bioinformatics 39 (7) (2023) btad432.

[20] L. Jiang, C. Xu, Y. Bai, A. Liu, Y. Gong, Y.-P. Wang, H.-W. Deng, Autosurv: interpretable deep learning framework for cancer survival analysis incorporating clinical and multi-omics data, NPJ Precision Oncology 8 (1) (2024) 4.

[21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, Nature Reviews Physics 3 (6) (2021) 422–440.

[22] B. E. Tropp, Principles of molecular biology, Jones & Bartlett Publishers, 2012.

[23] M. Špendl, Nov način optimizacije rabe kodonov z uporabo strojnega učenja na osnovi struktur proteinov, Ph.D. thesis, University of Ljubljana, Faculty of Chemistry and Chemical Technology (2022).

[24] Z. Wang, M. Gerstein, M. Snyder, RNA-Seq: a revolutionary tool for transcriptomics, Nature Reviews Genetics 10 (1) (2009) 57–63.

[25] X. Li, C.-Y. Wang, From bulk, single-cell to spatial RNA sequencing, International Journal of Oral Science 13 (1) (2021) 36.

[26] D. Chicco, Siamese neural networks: An overview, Artificial Neural Networks (2021) 73–94.

[27] R. Balestriero, M. Ibrahim, V. Sobal, A. S. Morcos, S. Shekhar, T. Goldstein, F. Bordes, A. Bardes, G. Mialon, Y. Tian, A. Schwarzschild, A. G. Wilson, J. Geiping, Q. Garrido, P. Fernandez, A. Bar, H. Pirsiavash, Y. LeCun, M. Goldblum, A Cookbook of Self-Supervised Learning, ArXiv abs/2304.12210 (2023).

[28] D. R. Cox, The regression analysis of binary sequences, Journal of the Royal Statistical Society: Series B (Methodological) 20 (2) (1958) 215–232.

[29] K. P. Murphy, Probabilistic machine learning: an introduction, MIT press, 2022.

[30] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning internal representations by error propagation, parallel distributed processing, explorations in the microstructure of cognition, Biometrika 71 (599-607) (1986) 6.

[31] S. Kullback, R. A. Leibler, On information and sufficiency, The Annals of Mathematical Statistics 22 (1) (1951) 79–86.

[32] T. G. Clark, M. J. Bradburn, S. B. Love, D. G. Altman, Survival analysis part i: basic concepts and first analyses, British Journal of Cancer 89 (2) (2003) 232–238.

[33] M. J. Bradburn, T. G. Clark, S. B. Love, D. G. Altman, Survival analysis part ii: multivariate data analysis–an introduction to concepts and methods, British Journal of Cancer 89 (3) (2003) 431–436.

[34] M. J. Bradburn, T. G. Clark, S. B. Love, D. G. Altman, Survival analysis part iii: multivariate data analysis–choosing a model and assessing its adequacy and fit, British Journal of Cancer 89 (4) (2003) 605–611.

[35] T. G. Clark, M. J. Bradburn, S. B. Love, D. Altman, Survival analysis part iv: further concepts and methods in survival analysis, British Journal of Cancer 89 (5) (2003) 781–786.

[36] K.-M. Leung, R. M. Elashoff, A. A. Afifi, Censoring issues in survival analysis, Annual Review of Public Health 18 (1) (1997) 83–104.

[37] E. L. Kaplan, P. Meier, Nonparametric estimation from incomplete observations, Journal of the American Statistical Association 53 (282) (1958) 457–481.

[38] M. Schumacher, G. Bastert, H. Bojar, K. Hübner, M. Olschewski, W. Sauerbrei, C. Schmoor, C. Beyerle, R. Neumann, H. Rauschecker, Randomized 2 x 2 trial evaluating hormonal treatment and the duration of chemotherapy in node-positive breast cancer patients. German Breast Cancer Study Group., Journal of Clinical Oncology 12 (10) (1994) 2086–2093.

[39] D. Collett, Modelling survival data in medical research, Chapman and Hall/CRC, 2023.

[40] G. Gašparac, Development of a surrogate endpoint for predicting graft failure in kidney transplant patients, Ph.D. thesis, University of Ljubljana, Faculty of Computer and Information Science (2022).

[41] D. R. Cox, Regression models and life-tables, Journal of the Royal Statistical Society: Series B (Methodological) 34 (2) (1972) 187–202.

[42] L. Mariani, D. Coradini, E. Biganzoli, P. Boracchi, E. Marubini, S. Pilotti, B. Salvadori, R. Silvestrini, U. Veronesi, R. Zucali, et al., Prognostic factors for metachronous contralateral breast cancer: a comparison of the linear Cox regression model and its artificial neural network extension, Breast Cancer Research and Treatment 44 (1997) 167–178.

[43] A. Xiang, P. Lapuerta, A. Ryutov, J. Buckley, S. Azen, Comparison of the performance of neural network methods and Cox regression for

censored survival data, Computational Statistics & Data Analysis 34 (2) (2000) 243–257.

[44] D. J. Sargent, Comparison of artificial neural networks with other statistical approaches: results from medical data sets, Cancer: Interdisciplinary International Journal of the American Cancer Society 91 (S8) (2001) 1636–1642.

[45] J. L. Katzman, U. Shaham, A. Cloninger, J. Bates, T. Jiang, Y. Kluger, DeepSurv: personalized treatment recommender system using a Cox proportional hazards deep neural network, BMC Medical Research Methodology 18 (2018) 1–12.

[46] C. Lee, W. Zame, J. Yoon, M. Van Der Schaar, Deephit: A deep learning approach to survival analysis with competing risks, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, 2018.

[47] J. Wang, N. Chen, J. Guo, X. Xu, L. Liu, Z. Yi, SurvNet: A novel deep neural network for lung cancer survival analysis with missing values, Frontiers in Oncology 10 (2021) 588990.

[48] A. Tarkhan, N. Simon, BigSurvSGD: Big survival data analysis via stochastic gradient descent, arXiv preprint arXiv:2003.00116 (2020).

[49] T. Chen, H. L. He, G. M. Church, Modeling gene expression with differential equations, in: Biocomputing'99, World Scientific, 1999, pp. 29–40.

[50] A. Paszke, S. Gross, F. Massa *et al.*, PyTorch: An Imperative Style, High-Performance Deep Learning Library, in: Advances in Neural Information Processing Systems 32, Curran Associates, Inc., 2019, pp. 8024–8035.

[51] M. Stražar, M. Žitnik, B. Zupan, J. Ule, T. Curk, Orthogonal matrix factorization enables integrative analysis of multiple RNA binding proteins, Bioinformatics 32 (10) (2016) 1527–1535.

[52] M. Ghandi, et al., Next-generation characterization of the cancer cell line encyclopedia, Nature 569 (7757) (2019) 503–508.

[53] W. Yang, J. Soares, P. Greninger, E. J. Edelman, H. Lightfoot, S. Forbes, N. Bindal, D. Beare, J. A. Smith, I. R. Thompson, et al., Genomics of drug sensitivity in cancer (gdsc): a resource for therapeutic biomarker discovery in cancer cells, Nucleic Acids Research 41 (D1) (2012) D955–D961.

[54] C. Curtis, S. P. Shah, S.-F. Chin, G. Turashvili, O. M. Rueda, M. J. Dunning, D. Speed, A. G. Lynch, S. Samarajiwa, Y. Yuan, et al., The genomic and transcriptomic architecture of 2,000 breast tumours reveals novel subgroups, Nature 486 (7403) (2012) 346–352.

[55] M. Rahman, L. K. Jackson, W. E. Johnson, D. Y. Li, A. H. Bild, S. R. Piccolo, Alternative preprocessing of RNA-Sequencing data in The Cancer Genome Atlas leads to improved analysis results, Bioinformatics 31 (22) (2015) 3666–3672.

[56] L. Vidman, D. Källberg, P. Rydén, Cluster analysis on high dimensional RNA-seq data with applications to cancer research-an evaluation study, PLoS One 14 (12) (2019) e0219102.

[57] A. Subramanian, et al., A next generation connectivity map: L1000 platform and the first 1,000,000 profiles, Cell 171 (6) (2017) 1437–1452.

[58] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2019.

[59] P. A. Moran, Notes on continuous stochastic phenomena, Biometrika 37 (1/2) (1950) 17–23.

[60] L. Van der Maaten, G. Hinton, Visualizing data using t-sne., Journal of Machine Learning Research 9 (11) (2008).

[61] J. Demšar, Statistical comparisons of classifiers over multiple data sets, The Journal of Machine Learning Research 7 (2006) 1–30.